

RAGN: Detecting unknown malicious network traffic using a robust adaptive graph neural network

Ernest Akpaku^a, Jinfu Chen^{a,b,*}, Mukhtar Ahmed^a, Francis Kwadzo Agbenyegah^c, William Leslie Brown-Acquaye^c

^a School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China

^b Jiangsu Key Laboratory of Security Technology for Industrial Cyberspace, Jiangsu University, 301 Xuefu Road, Zhenjiang, 212013, Jiangsu, China

^c Department of Information Technology, Ghana Communication Technology University, Accra, Ghana

ARTICLE INFO

Keywords:

Network security
Dynamic graph
Network anomalies
Robust graphs
Intrusion defense
Edge attention

ABSTRACT

As network environments evolve, detecting unknown malicious network traffic becomes increasingly challenging due to the dynamic and sophisticated nature of modern cyberattacks. Graph Attention Networks (GATs) have shown promise in modeling complex network interactions but remain vulnerable to adversarial attacks that exploit weaknesses in the graph structure. In this work, we propose the Robust Adaptive Graph Neural Network (RAGN), an enhanced GAT-based framework that introduces adaptive attention mechanisms to improve detection accuracy and robustness against adversarial manipulations in network traffic graphs. RAGN iteratively adjusts the graph structure and feature space to suppress adversarial perturbations by assigning lower attention scores to unreliable edges and refining feature representations based on the feature smoothness regularization principle. To assess the robustness of the proposed RAGN model and compare it with baseline models, we introduced an effective dynamic graph attack method known as Semantic-Preserving Adversarial Node Injection (SPAN). We benchmarked its performance against state-of-the-art graph attack methods, including DICE, DGA, and RWCS. SPAN incrementally injects small batches of malicious nodes, refining their edges and features to target both the structural and temporal aspects of dynamic graphs. It preserves semantic integrity, and generates effective yet imperceptible perturbations, providing a rigorous test of the resilience of graph neural networks. Experiments conducted on four datasets, demonstrate that RAGN demonstrates robustness against adversarial, and zero-day attacks. It also demonstrates resilience against targeted, malicious node injection attacks in dynamic network environments. RAGN demonstrated consistent robustness, with misclassification rates increasing only marginally (by less than 1.2%) even under significant dynamic changes.

1. Introduction

The detection of unknown malicious network traffic, especially in the face of adversarial attacks, presents significant challenges in modern network environments [1]. As networks grow in scale and complexity, traditional intrusion detection methods struggle to cope with dynamic and sophisticated threats, including zero-day attacks and adversarial manipulations designed to evade detection systems [2]. Modern networks face an escalating wave of sophisticated cyberattacks that exploit their increasing complexity and dynamic nature. Existing network attack detection approaches often represent network traffic using Euclidean structures, which fail to capture the intricate correlations between different traffic flows [3]. Graph-based models, particularly Graph Attention Networks (GATs) [4], have demonstrated potential in

modeling such complex relationships. By leveraging attention mechanisms [5] to weigh the importance of various connections between network entities, GATs have achieved notable success in node classification and malicious traffic detection. However, GATs and similar graph neural networks are inherently vulnerable to adversarial attacks, where an attacker can manipulate the graph structure or features by introducing deceptive edges or perturbing node characteristics [6]. Most current methods rely on static graph structures, which fail to account for the dynamic and time-evolving nature of real-world networks. This static nature limits their adaptability to new threats and changing environments. Adversarial attacks, such as edge manipulation and feature perturbation, significantly degrade the performance of graph-based models. Existing approaches lack robust mechanisms to mitigate these adversarial effects, leaving networks vulnerable. For

* Corresponding author.

E-mail address: jinfuchen@ujs.edu.cn (J. Chen).

<https://doi.org/10.1016/j.comnet.2025.111184>

Received 29 October 2024; Received in revised form 23 February 2025; Accepted 2 March 2025

Available online 10 March 2025

1389-1286/© 2025 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

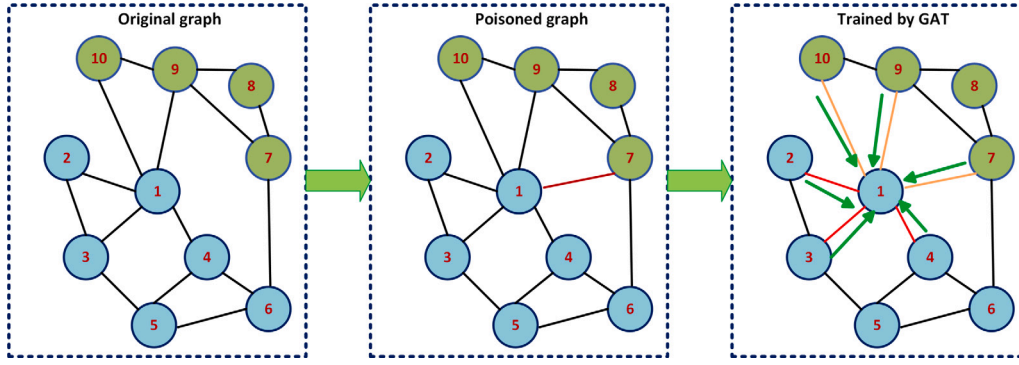


Fig. 1. Original graphs can be attacked by the negative edges between nodes 1 and 7 with different labels. The update function of node 1 aggregates more negative information from nodes 7, 9, 10 than positive information from nodes 2, 3, 4.

instance, in traditional GNNs, as shown in Fig. 1, the aggregation function in GATs process information from various neighbors for a given node. This aggregated information can be broadly classified into two categories: positive and negative edges. Positive edges represent connections between nodes with similar labels, which are beneficial for accurately updating the node's feature representation. In contrast, negative edges represent connections between nodes with differing labels, which can introduce noise and hinder the feature update process [7]. Adversarial edge attacks exploit this mechanism by either adding deceptive positive edges or reducing critical negative edges, disrupting the balance of aggregated information [8]. This imbalance diminishes the model's ability to accurately distinguish between benign and malicious nodes, ultimately undermining its robustness in dynamic and adversarial network environments.

These adversarial actions significantly degrade the performance of GATs, particularly when deployed in dynamic environments [9]. Such actions can either increase the influence of nodes with similar labels (positive edges) or decrease the influence of nodes with differing labels (negative edges), severely degrading the GAT's performance [7]. In a network traffic graph, nodes represent entities like IP addresses or devices, and edges represent traffic flows or connections between these entities. Malicious actors can simulate traffic between entities that do not typically communicate, creating false connections in the graph [10]. This can make benign nodes appear malicious or vice versa, potentially leading to false positives or negatives in threat detection [11]. Conversely, an attacker might remove existing edges from the graph. For example, severing connections that signify normal interactions between critical network services could hide ongoing malicious activities that disrupt these usual patterns. These manipulations aim to alter the “view” that the GAT has over the network, misleading it about the true nature of the network's activity [12]. Second, each node in a network traffic graph can have features like the number of packets, types of protocols used, or frequency of connections [13]. Adversarial attacks might involve an attacker changing these features to mask malicious traffic as normal or make normal activity appear suspicious, such as altering the volume of traffic or the frequency of connections to simulate a denial-of-service attack or to hide such an attack among regular traffic spikes. These feature modifications can undermine the effectiveness of GATs, as the attention mechanism within the GAT could assign incorrect importance to altered nodes, leading to flawed threat assessments.

Specifically, three problems exist, (1) Static Attention Mechanism: The original GAT applies a fixed attention mechanism that, while effective in learning from graph data, does not dynamically adapt to changes in the graph's structure or node features over time [5]. This could be less effective in environments where adversarial tactics continuously evolve. (2) Lack of Iterative Refinement: GAT processes the graph based on the data available at the time of training, without iterative adjustments post-deployment unless retrained. This static nature makes

it less suitable for environments where network behavior and threats evolve dynamically, as is common in cybersecurity [8]. (3) Vulnerability to Adversarial Attacks: While GATs are powerful in weighting the importance of different nodes, they do not inherently distinguish between manipulations meant to deceive the attention mechanism and genuine data variations [7]. This makes them potentially vulnerable to adversarial attacks targeting their attention mechanisms by mimicking benign characteristics or subtly altering connections.

To address these limitations, we propose the Robust Adaptive Graph Network (RAGN), an enhanced GAT-based model that introduces key innovations aimed at improving both detection accuracy and robustness against adversarial attacks. RAGN iteratively adjusts the graph structure and node features, leveraging dynamic attention mechanisms to mitigate the impact of adversarial manipulations. The model is further strengthened through the application of Laplacian regularization, which enforces feature smoothness across the graph and enhances resilience against adversarial edge and feature perturbations. Our work proposes defensive strategies that increase the positive effects and minimize the negative impacts of neighbor relationships in the network graph. We note that, in many real-world scenarios, not all nodes will have labels indicating their nature (benign or malicious), especially in dynamic networks where new nodes (e.g., devices, IP addresses) frequently appear. Refining the attention mechanism in GATs to weigh the importance of a node's features based on the aggregated information from its neighbors, can help in inferring the labels implicitly. We also note that GATs, by design, weigh nodes based on their relevance to a task, but if the input data (graph structure or node features) is manipulated, the attention mechanism may inaccurately prioritize malicious nodes or connections [14].

As such, we introduce smooth feature representations across the graph based on Laplacian regularization to reduce the model's sensitivity to small changes in the graph structure. Also, we introduce an extra dynamic score to adjust the attention effects for different edges during the training procedure in response to perceived anomalies or shifts in network traffic patterns that could indicate adversarial activities. We aim to design an improved GAT for malicious network traffic detection while defending against adversarial attacks based on graph information. Therefore, the key to our approach is the development of innovative methods to: Distinguish between different types of neighbor connections without extensive labeling; and design attention scores that adaptively enhance the robustness of the network against adversarial manipulations.

Next, we note, that in network traffic analysis, manipulating legitimate entities is challenging, making malicious node injection a more practical and stealthy attack strategy. Adversaries exploit the dynamic nature of network environments by adding malicious nodes that blend into evolving graph structures, making detection difficult. The performance of Dynamic Graph Neural Networks (DGNNs) depends heavily on the graph structure at any given time, and adversaries can

disrupt this by targeting optimal moments for attack using periodic decision strategies.

To address these challenges, we also introduce the Semantic-Preserving Adversarial Node Injection (SPAN) method. SPAN leverages graph vulnerabilities to inject malicious nodes at critical times, selecting targets based on structural fragility, connecting malicious nodes in batches, and using smoothing functions to mimic legitimate behaviors. This approach improves attack efficiency and also minimizes detection risk. Evaluated on dynamic graph datasets, SPAN outperforms existing attack methods, demonstrating superior efficiency in degrading DGNN performance. Thus, SPAN provides a powerful and comprehensive attack framework for evaluating the resilience of GNN-based detection models against zero-day threats, dynamic challenges, and adversarial manipulations. The contributions of our method are summarized as follows:

1. We introduce a dynamic attention mechanism that progressively increases attention scores for edges in GATs, enabling more effective detection and defence against zero-day malicious traffic. This mechanism differentiates between benign and adversarial edges based on the feature smoothness assumption, improving the model's ability to detect subtle adversarial manipulations.
2. An iterative refinement process is introduced into the training phase of GATs, allowing continuous adjustment of both the graph structure and node features. This process optimizes the allocation of attention scores, progressively prioritizing real edges over adversarial ones, resulting in enhanced model accuracy and robustness against various attacks.
3. To further strengthen defence against adversarial edges, we incorporate an extra attention-scoring component that assigns lower scores to potentially harmful edges, mitigating their impact on the overall model performance.
4. We extended the scope of graph adversarial attacks to dynamic graph neural networks (DGNNs) by introducing a novel method, SPAN (Semantic-Preserving Adversarial Node Injection). SPAN addresses the limitations of previous approaches that were confined to static graph neural networks, offering an effective and adaptive strategy for attacking dynamic graph environments.
5. Our experimental results on the CICIDS-2018, CTU-13, USTC-TFC2016, UJS-IDS2022 datasets demonstrate that RAGN achieves superior performance in detecting and classifying malicious network traffic, even under various types of adversarial attacks, including targeted, dynamic node attacks, and random node and edge attacks. RAGN consistently outperforms state-of-the-art methods across these diverse attack scenarios, showcasing its robustness and effectiveness in real-world settings.

The remainder of this paper is organized as follows. Section 2 introduces the existing research related to malicious traffic detection and graph neural networks. In Section 3, we describe our proposed method, including the steps establishing the graph structure, and constructing the RAGN method. Section 4 presents the experimental setup details. Section 5 presents the results with a comparative analysis through experiments, evaluating our method against state-of-the-art methods. The paper concludes with Section 6, which summarizes our findings and outlines directions for future research.

2. Related studies

Graph-based learning techniques, particularly Graph Neural Networks (GNNs), have gained significant attention in recent years for network intrusion detection and traffic classification. Among these, Graph Convolutional Networks (GCNs) and Graph Attention Networks (GATs) have emerged as powerful tools for modeling network traffic and detecting malicious activities [15]. However, despite their success,

several limitations remain when it comes to defending against adversarial attacks and detecting unknown malicious traffic. The detection and defence against malicious network traffic, especially in the presence of adversarial attacks remains a major concern.

In recent years, the adoption of graph-based models has gained significant traction [16], as network traffic data naturally forms a graph structure where entities such as IP addresses, hosts, and devices communicate over edges representing network flows [13]. GNNs, particularly Graph Convolutional Networks (GCNs) introduced by Kipf and Welling (2017), revolutionized the field by efficiently propagating information across graph structures, enabling better generalization to traffic patterns. GCNs have been employed to detect anomalies in network traffic by modeling network entities and their interactions as nodes and edges, respectively [17]. In [18], the E-GraphSAGE model was introduced. It leverages the inherent structure of graph-based data to detect and mitigate network-based cyber attacks by capturing both edge features and topological information. The E-GraphSAGE algorithm was extensively evaluated on four recent NIDS benchmark datasets. The results demonstrated that E-GraphSAGE outperformed state-of-the-art methods in key classification metrics, showcasing its potential and robustness. Unlike traditional GCNs that may focus solely on node features, GCN-ETA [11] also considers the relationships between different flows, such as shared source and destination hosts. Compared to the bench-marked baselines in the work, the model improved accuracy rates, AUC, and F1-scores exceeding 98%, along with the capability to process over 1300 flows per second. In [10], the authors combined GCN with Multi-Head Self-Attention (MHSA) mechanism. The model addresses several limitations of existing GCN-based methods, such as incomplete feature descriptions and inadequate traffic correlation mechanisms. MHSA is integrated into the GCN to assign larger weights to more important features [19]. This enhanced the model's ability to focus on critical aspects of the data, improving detection stability and efficiency. The model showed an improvement of about 2.4% in accuracy, recall, and F1-measure, and about 2.1% in precision in comparison with other graph-based models.

Graph Attention Networks (GATs), proposed by Veličković et al. [4], improved upon GCNs by incorporating an attention mechanism that allows the model to focus on the most relevant neighbors during message passing. GATs have shown strong potential in network traffic classification tasks by giving higher importance to influential neighbors. The framework presented in [20] uses GAT and transformer to model the dependencies between different time series variables. The authors introduced a new type of graph convolution called Influence Propagation convolution is introduced to describe the anomaly information flow between network nodes. The results demonstrate that the proposed framework outperforms baseline alternative state-of-the-art methods on multiple benchmark datasets, showcasing its effectiveness in detecting malicious network traffic intrusions. However, despite their effectiveness, E-GraphSAGE, GCN-MHSA, GCN-ETA, and the many other GNN-GAT-GCN-based models [21–24] are limited by their inability to handle heterogeneous and dynamic graph structures, often leading to suboptimal detection of evolving malicious traffic.

The vulnerability of graph-based models to adversarial attacks has drawn significant attention in recent research. For instance, GCNs and GATs have shown to be susceptible to adversarial perturbations when challenged with attack methods such as Netattack [25], RLS2V [26], and metattack [27]. Meta-attack involves strategically adding or removing edges in the graph to degrade model performance. Netattack introduces the unnoticeable perturbations on both structures and features. RLS2V uses reinforcement learning to generate attacks on GNNs. The metattack parameterized the graph structure and used the gradient information to attack GAT. In these attacks, malicious actors introduce perturbations to the graph structure or node features with the aim of misleading the model and causing incorrect inferences. Such attacks can take various forms, such as the addition of adversarial nodes

designed to disrupt the graph's normal patterns and introduce anomalies. Attackers may tamper with the edges between nodes, altering the graph's connectivity either by inserting or removing edges, or by changing their weights [28]. This manipulation of edges can effectively disrupt the flow of information across the graph, undermining the GNN's ability to learn accurate representations of the data.

In addition to these direct modifications, attackers can also alter the features associated with individual nodes, effectively changing the input presented to the GNN and skewing its output towards incorrect conclusions [29]. More sophisticated structural changes can be implemented, such as the creation of clusters of malicious nodes or the isolation of certain nodes, which can significantly impact the graph's properties and the GNN's capacity to learn from the graph structure. These attacks highlight the vulnerability of GNNs to manipulations of their underlying graph structure and emphasize the need for robust defence mechanisms to safeguard against such threats.

To address these adversarial challenges, several defence mechanisms have been proposed. In [30], for instance, the authors argued that future IDS should track how attacks evolve across system layers by adopting attack graphs to overcome the limitations of current IDS that are unable to identify unseen attacks and access particular programs to confirm attacks [31]. An attack graph-based intrusion detection approach is proposed and a few challenges to the realization of their proposal are identified [32]. With two use cases, they showcase how to apply their approach. Overall, a conceptual overview of why and how attack graphs should be used for the next generation of IDS is presented. The authors evaluated their proposed approach through two case studies: attacks on file retrieval, such as Time-of-Check to Time-of-Use (TOCTOU) attacks, and attacks propagated among processes, such as attacks on Shellshock vulnerabilities. The concept is implemented in [21] where the GNN-IDS is proposed. In GNN-IDS, an attack graph and real-time measurement mechanisms that represent static and dynamic attributes of computer networks, respectively, are incorporated and associated to represent complex computer networks. The authors demonstrate the robustness of GNN-IDS against adversarial attacks by evaluating the model's performance on datasets with added Gaussian noise, simulating changes in network conditions.

The results show that GNN models produce fewer false positives than traditional models, indicating their effectiveness in avoiding detection evasions. However, the method is not tested in defending against targeted and non-targeted attacks, particularly in scenarios where the graph structure is heavily perturbed. Another method is TCGNN [33] which transforms network packets into an undirected graph, and then adopts a two-layer GCN with three different aggregation strategies (Mean, Attention, and LSTM). Adversarial attacks would need to alter these multiple features to be effective, which is more complex than attacking models that rely on a single type of feature. The results show that TCGNN can identify unknown network packets with an extremely high accuracy rate over the existing packet-grained traffic classification methods. EdgeTorrent [23] defends against adversarial attacks by leveraging real-time temporal graph representations, adversarial training, and a streaming architecture that enables rapid adaptation and detection of anomalies. According to the authors, EdgeTorrent is the first IDS to perfectly classify the StreamSpot dataset, which includes various types of network traffic, indicating its robustness against various forms of adversarial evasion techniques.

Despite these advancements, none of these works focus on the robustness principle of GAT's aggregation function. In typical GATs, the aggregation function collects information from neighboring nodes to update the feature representation of the central node. Generally, the information aggregated by GAT can be classified into two categories: (1) information from neighboring nodes with the same label, which is beneficial for updating positive edges, and (2) information from neighboring nodes with different labels, which has a negative impact on the node's feature update. In adversarial scenarios, attackers can manipulate these relationships by adding additional positive edges or

reducing negative edges, thereby disrupting the balance of information aggregation. This manipulation, as shown in Table 3, can mislead the model into increasing the influence of harmful neighbors or decreasing the weight of beneficial ones. Similarly, adversarial feature attacks can degrade the node's feature information by tampering with nodes of the same label, making it harder for the model to distinguish between benign and malicious entities.

The primary gap in these existing approaches is their inability to dynamically adjust to adversarial changes in both the graph structure and feature space. Most models employ static attention mechanisms that cannot distinguish between benign and adversarial edges effectively, nor can they adjust in real-time to adversarial manipulations [34]. Furthermore, they lack iterative refinement processes that allow the model to adapt to evolving attack strategies. To address these limitations, RAGN introduces a dynamic attention mechanism that adjusts edge scores based on both the structure and features of the graph, coupled with iterative refinement during training. This ensures that the model can continuously prioritize real edges over adversarial ones and enhance detection accuracy, even under adversarial conditions such as zero-day attacks. By incorporating these innovations, RAGN offers a significant improvement in both detection accuracy and robustness compared to the aforementioned models. In addition, RAGN also adjusts the influence of both positive and negative neighbors by applying a dynamic attention mechanism to the edges, based on an extra attention score computed using a feature smoothness regularization mechanism. This dynamic attention score is generated progressively and plays a critical role in improving the model's robustness for detecting and defending against unknown malicious network traffic. Next, RAGN refines the node features to suppress feature noise, which can be exploited by attackers to make malicious traffic appear benign. This step ensures that the model remains effective in identifying hidden patterns within dynamic and evolving network environments, even in the presence of adversarial noise. The dynamic attention score is then applied to the edges, adjusting the importance of connections between nodes. This score is generated using the adjusted edge weights and helps mitigate the impact of adversarial attacks. Specifically, RAGN reduces the influence of adversarial edges (malicious traffic connections) by assigning lower attention to them, while giving higher attention to reliable, benign connections. This iterative refinement of both the edge weights and the node features enhances the model's ability to detect unknown malicious traffic while remaining robust against adversarial manipulations.

3. Preliminaries

This section outlines the theoretical motivations, provides an analysis, and delves into the foundational theory of Graph Attention Networks (GAT) that underpin our study.

3.1. Notations

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph representing the network traffic, with $\mathcal{V}$ being the set of vertices (network entities) and $\mathcal{E}$ being the set of edges (interactions between entities). Each graph can be represented by the adjacency matrix of the graph $\mathbf{A} \in \mathbb{R}^{N \times N}$ where A_{ij} indicates the presence of an interaction from entity i to entity j . Let $\mathbf{X} \in \mathbb{R}^{N \times D}$ be the feature matrix, where each row represents the features of an entity and D is the number of features. The label matrix is expressed as $\mathbf{Y} \in \mathbb{R}^{N \times K}$ where each row represents the labels (benign or malicious) of an entity and K is the number of label categories. For Neighborhood and Aggregation, we let $\mathcal{N}(v_i)$ be the set of neighbors of entity v_i , representing direct interactions. $\mathbf{A}^{(k)}$ as the k th power of the adjacency matrix $\mathbf{A}$, representing k -hop neighborhood interactions. For the attention Mechanism, e_{ij} represents the attention coefficient between entities v_i and v_j , capturing the importance of one entity's features to another. The feature matrix after l th layer of attention mechanism is

$\mathbf{H}^{(l)} \in \mathbb{R}^{N \times M}$ where M is the dimension of the output space. For dynamic Graph Adjustment, $\tilde{\mathbf{A}}$ is the dynamically adjusted adjacency matrix, incorporating robustness against adversarial attacks. While s_{ij} is the similarity score between entities v_i and v_j , used to update the graph structure. The perturbation matrix representing adversarial edge additions or deletions is expressed as $\Delta\mathbf{A}$ for adversarial attacks. Let $\Delta\mathbf{X}$ be the perturbation matrix representing adversarial feature modifications. On optimization and Learning Θ is the set of model parameters to be learned. $\mathcal{L}(\Theta)$ is the loss function that depends on the model parameters Θ . $\mathcal{L}_{\text{adv}}$ is the adversarial loss component, focusing on robustness. We consider a semi-supervised node classification problem, where only a subset of nodes $\mathcal{V}_l \subset \mathcal{V}$ are labeled with labels $\mathcal{Y}_l$.

3.2. Graph Attention Network (GAT)

Graph Attention Networks (GAT) update the features of a node by aggregating information from its neighbors, using dynamically computed attention scores to assign varying importance to different edges. This mechanism allows GAT to effectively capture task-specific dependencies in graph structures, even in heterogeneous and noisy environments. In GAT, the updated feature of a node v at layer $k+1$, denoted as $\mathbf{x}_v^{(k+1)}$, is computed by aggregating information from its neighbors $\mathcal{N}(v)$, weighted by learned attention scores $\alpha_{uv}^{(k)}$. The operation is expressed as:

$$\mathbf{x}_v^{(k+1)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv}^{(k)} \mathbf{W}^{(k)} \mathbf{x}_u^{(k)} \right), \quad (1)$$

where σ is a non-linear activation function; $\mathcal{N}(v)$ is the set of neighbors of node v ; $\alpha_{uv}^{(k)}$ is the attention coefficient for the edge connecting nodes u and v at layer k ; $\mathbf{W}^{(k)} \in \mathbb{R}^{d_{k+1} \times d_k}$ is the learnable weight matrix for layer k ; $\mathbf{x}_u^{(k)} \in \mathbb{R}^{d_k}$ is the feature vector of node u at layer k . The attention coefficient $\alpha_{uv}^{(k)}$ determines the relative importance of node u 's information to node v . It is computed using:

$$\alpha_{uv}^{(k)} = \frac{\exp \left(\text{LeakyReLU} \left(\mathbf{a}^\top \left[\mathbf{W}^{(k)} \mathbf{x}_u^{(k)} \parallel \mathbf{W}^{(k)} \mathbf{x}_v^{(k)} \right] \right) \right)}{\sum_{w \in \mathcal{N}(v)} \exp \left(\text{LeakyReLU} \left(\mathbf{a}^\top \left[\mathbf{W}^{(k)} \mathbf{x}_w^{(k)} \parallel \mathbf{W}^{(k)} \mathbf{x}_v^{(k)} \right] \right) \right)}, \quad (2)$$

where: $\mathbf{a} \in \mathbb{R}^{2d_k}$ is a learnable vector that determines the attention mechanism, $\parallel$ denotes concatenation of feature vectors, and LeakyReLU introduces non-linearity for better representation of feature interactions. The softmax normalization ensures that the attention scores $\alpha_{uv}^{(k)}$ are non-negative and sum to one for all neighbors of node v , ensuring stability. To enhance the model's expressiveness, GAT employs multi-head attention, where multiple independent attention mechanisms compute attention scores. The features computed by each attention head are then concatenated to form the final node representation. At layer $k+1$, the updated feature of node v is:

$$\mathbf{x}_v^{(k+1)} = \parallel_{m=1}^M \sigma \left(\sum_{u \in \mathcal{N}(v)} \alpha_{uv}^{(k,m)} \mathbf{W}^{(k,m)} \mathbf{x}_u^{(k)} \right) \quad (3)$$

where M is the number of attention heads, $\parallel$ denotes concatenation, and $\alpha_{uv}^{(k,m)}$ and $\mathbf{W}^{(k,m)}$ are the attention scores and weight matrices for the m th head, respectively.

Multi-head attention improves the robustness and generalization of the model by aggregating information from multiple perspectives, allowing the model to capture diverse patterns in node relationships. The learnable parameters of GAT are collectively denoted by $\Theta = \{\mathbf{W}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{a}\}$. Given a classification task, the model learns a function $f_{\text{GAT}} : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times C}$, where C is the number of classes. The classification probabilities for labeled nodes are predicted using a cross-entropy loss:

$$\mathcal{L}_{\text{GAT}}(\Theta, A, X, Y) = - \sum_{v \in \mathcal{V}_L} \sum_{c=1}^C Y_{v,c} \log (f_{\text{GAT}}(A, X)_{v,c}), \quad (4)$$

where $\mathcal{V}_L$ is the set of labeled nodes, $Y_{v,c}$ is a binary indicator of whether node v belongs to class c , and $f_{\text{GAT}}(A, X)_{v,c}$ is the predicted probability of node v being in class c . The attention mechanism in

Table 1

GAT accuracy on synthetic graphs with varying positive to negative edge ratios.

Graph	P_i	P_{ii}	E_1	E_2	E_1/E_2	GAT accuracy (%)
1	0.02	0.01	5,982	5,224	1.22	53.60
2	0.04	0.01	10,894	5,770	2.05	81.25
3	0.06	0.01	15,504	7,076	2.91	96.37
4	0.08	0.01	20,354	5,634	3.99	99.88

GAT enables the model to capture the relative importance of different edges dynamically, improving its performance on heterogeneous and noisy graphs. Multi-head attention further enhances robustness and expressiveness by leveraging multiple perspectives, making GAT highly effective for graph-based learning tasks.

3.3. Motivation

This section serves to (1) define the baseline capabilities of GAT, (2) identify its limitations in dynamic and adversarial settings, (3) establish the need for enhancements, and (4) lay the groundwork for a comparative evaluation with the proposed RAGN model. As discussed in Section 2 (related studies), GATs exhibit limitations that can hinder their effectiveness, particularly under adversarial conditions. For instance, in Fig. 1, the aggregation function consolidates information from various neighboring nodes for node 1. This aggregated information can typically be categorized into two types. The first type is the information from neighbors that share the same label, which aids in updating the node's feature information, and these connections are referred to as positive edges. The second type consists of information from neighboring nodes with differing labels, which can hinder the process of updating node features, and these are called negative edges. Adversarial edge attacks can either introduce more positive edges or reduce the negative edges, thus affecting the balance between these two types of aggregated data. Similarly, adversarial feature attacks can degrade the quality of information from nodes that share the same label.

To explore these limitations, we conducted empirical analyses using both synthetic and open datasets, investigating how the ratio of positive to negative edges (E_1/E_2) influences GAT's performance. We employed the *Stochastic Block Model (SBM)* [35] to generate synthetic graphs, allowing precise control over intra-class and inter-class connection probabilities. This control was essential for manipulating the E_1/E_2 ratio and observing its impact on GAT's performance. For graph construction, we created $N = 1000$ nodes, equally divided into two classes with class 1 representing benign network entities and class 2 representing malicious network entities, for the edge formation intra-class connection probability (P_i) is defined as the probability that two nodes within the same class are connected. Inter-class connection probability (P_{ii}) is also defined as the probability that two nodes from different classes are connected. By varying P_i while keeping P_{ii} constant, we manipulated the E_1/E_2 ratio to create graphs with different proportions of positive and negative edges. Specifically, we set $P_{ii} = 0.01$ and increased P_i from 0.02 to 0.08. We generated four synthetic graphs with varying E_1/E_2 ratios and evaluated GAT's performance on each. For each graph, 10% of the nodes were used for training and another 10% for testing. Table 1 presents the parameters and results for each synthetic graph.

As observed, when the E_1/E_2 ratio is close to 1 (Graph 1), GAT's accuracy is around 50%, similar to random guessing. This indicates that when the number of positive and negative edges is nearly equal, GAT struggles to make accurate classifications. However, as P_i increases and the E_1/E_2 ratio rises, GAT's accuracy improves significantly. In Graph 4, where $E_1/E_2 \approx 4$, the model achieves near-perfect accuracy.

To validate our findings, we conducted experiments on four network traffic datasets: *CICIDS-2018*, *CTU-13*, *USTC-TFC2016*. These datasets inherently possess different E_1/E_2 ratios due to the nature of network

Table 2

GAT accuracy on real-world datasets with different positive to negative edge ratios.

Dataset	E_1	E_2	E_1/E_2	GAT accuracy (%)
CICIDS-2018	10,687	5,986	4.10	83.97
CTU-13	25,402	6,934	2.79	75.26
USTC-TFC2016	30,278	3,150	9.61	96.33

Table 3

GAT accuracy on CICIDS-2018 under different perturbation rates.

Perturbation rate (%)	E_1	E_2	E_1/E_2	GAT accuracy (%)
0	10,152	1,986	4.10	83.97
5	10,232	2,412	3.41	80.44
10	9,864	2,840	2.92	75.61
15	10,453	3,332	2.49	69.78
20	10,358	3,852	2.13	59.94
25	10,862	4,240	1.95	54.78

interactions they represent. We constructed graphs from these datasets by representing network entities (e.g., IP addresses, devices) as nodes and interactions (e.g., communication events) as edges. Classes were defined based on the labeling of traffic as benign or malicious. Table 2 summarizes the results from these datasets.

The results mirror the trends observed in the synthetic data. Datasets with higher E_1/E_2 ratios, such as USTC-TFC2016, exhibit higher GAT accuracies. This consistency reinforces the conclusion that the proportion of positive to negative edges is a critical factor influencing GAT's performance. We further assessed GAT's robustness under adversarial conditions by simulating attacks on the CICIDS-2018 dataset. We manipulated the E_1/E_2 ratio by adding edges between nodes of different classes (increasing E_2) and removing edges between nodes of the same class (decreasing E_1). Adversarial perturbations were introduced using the RWCS (Random Walk Column Sum attack) method [36].

Table 3 shows the effect of increasing perturbation rates on GAT's performance. As the perturbation rate increased, the E_1/E_2 ratio decreased, and GAT's accuracy declined significantly. At a 25% perturbation rate, the accuracy dropped to approximately 55%, highlighting GAT's vulnerability to adversarial manipulation of the graph structure.

These experiments reveal key limitations of GAT in malicious network traffic detection. First, GAT's performance is highly sensitive to the E_1/E_2 ratio. A lower ratio, indicative of more negative edges, leads to substantial degradation in accuracy. Second, GAT's attention mechanism does not effectively differentiate between positive (intra-class) and negative (inter-class) edges, resulting in the aggregation of misleading information from malicious nodes. Third, GAT lacks mechanisms to assess the reliability of edges, making it susceptible to adversarial perturbations that alter the graph structure and mislead the attention mechanism. Our empirical analysis demonstrates that GAT's performance in malicious network traffic detection is significantly affected by the proportion of positive to negative edges in the graph. Adversarial attacks that manipulate the E_1/E_2 ratio can severely degrade the model's accuracy. The subsequent sections will delve into the architectural details of RAGN and present empirical evaluations showcasing its effectiveness compared to traditional GAT models.

3.4. Theoretical analysis

In this section, we delve into the theoretical underpinnings of the RAGN framework, which builds upon GAT by introducing a dynamic attention mechanism and a revised adjacency matrix. These mechanisms are designed to address the challenges posed by adversarial perturbations in graph-structured data, particularly in the context of network traffic analysis. Our approach incorporates both graph theory principles and neural network operations on graphs, focusing on the role of node features, graph topology, and the iterative refinement of the adjacency matrix to enhance robustness.

Stability of the attention mechanism. The dynamic attention mechanism in RAGN plays a crucial role in adapting the model's focus based on the evolving structure of the graph. In dynamic environments, such as adversarial settings, it is essential that the model remains stable under minor fluctuations in the graph's topology or node features. Stability ensures that small perturbations do not cause disproportionate changes in the model's outputs, which is crucial for maintaining performance consistency in the presence of noise or minor data variations.

Proposition 1. *The attention coefficients in RAGN are stable under perturbations to node features and graph topology, a crucial property for maintaining performance in adversarial settings.*

Proof. Consider the attention score $\alpha_{uv}^{(k)}$ for a graph edge (u, v) in the k th attention layer, computed as shown in Eq. (3). where W is the weight matrix, h_u and h_v are the feature vectors for nodes u and v , a is a shared attention vector, and $\mathcal{N}_u$ denotes the neighbors of node u . Now, assume a small perturbation Δh_v is applied to the feature vector h_v . The change in $\alpha_{uv}^{(k)}$ can be approximated using a first-order Taylor expansion:

$$\Delta \alpha_{uv}^{(k)} \approx \frac{\partial \alpha_{uv}^{(k)}}{\partial h_v} \Delta h_v$$

The partial derivative $\frac{\partial \alpha_{uv}^{(k)}}{\partial h_v}$ is bounded by the norms of a and $W^{(k)}$, ensuring that changes in $\alpha_{uv}^{(k)}$ are proportional to and bounded by $\|\Delta h_v\|$. This proves that the attention coefficients remain stable under small perturbations, as the changes in attention scores are linearly proportional to the perturbation in node features, preventing large fluctuations in the model's output.

Convergence of the dynamic adjustment. For RAGN to be effective, the iterative process of updating adjacency matrices and node features must converge to a stable configuration. Convergence ensures that the model reaches a state where further updates do not significantly alter its behavior, providing reliability and consistency in predictions. The iterative process enables the model to gradually refine the graph structure and focus on the most relevant features, which is particularly important in adversarial settings.

Proposition 2. *The iterative process for updating adjacency matrices and node features in RAGN converges to a stable state, enhancing the model's reliability.*

Proof. Consider the iterative update rule for the adjacency matrix A with a learning rate η :

$$A^{(t+1)} = A^{(t)} - \eta \nabla_A \mathcal{L}(A, X)$$

where $\mathcal{L}(A, X)$ is the loss function that includes a regularization term, such as Laplacian regularization, that promotes smooth feature transitions across edges. The gradient $\nabla_A \mathcal{L}(A, X)$ measures the impact of changes in A on the model's performance.

To prove convergence, we assume that the gradient $\nabla_A \mathcal{L}(A, X)$ is Lipschitz continuous, i.e., there exists a constant L such that:

$$\|\nabla_A \mathcal{L}(A_1, X) - \nabla_A \mathcal{L}(A_2, X)\| \leq L \|A_1 - A_2\|$$

This assumption ensures that the sequence $\{A^{(t)}\}$ is bounded and contractive. By the Banach Fixed-Point Theorem, since the sequence is contractive, it converges to a fixed point A^* , where:

$$\nabla_A \mathcal{L}(A^*, X) = 0$$

Thus, the adjacency matrix A converges to a stable configuration A^* , where further updates do not significantly alter the graph structure, and the model achieves optimal performance.

Derivation and iterative adjustment of the revised adjacency matrix. The revised adjacency matrix $\bar{A}$ plays a pivotal role in optimizing the graph structure for adversarial robustness. To address potential adversarial perturbations in the graph, the adjacency matrix is adjusted iteratively to reflect the most relevant nodes and edges while minimizing the influence of irrelevant or malicious changes.

Proposition 3. *The revised adjacency matrix $\bar{A}$ is adjusted iteratively to improve robustness to adversarial perturbations.*

Derivation: The adjustment of the adjacency matrix is performed by adding an update term $\Delta A^{(t)}$ to the original adjacency matrix A , as follows:

$$\bar{A}^{(t)} = A + \Delta A^{(t)}$$

At each iteration t , the update term $\Delta A^{(t)}$ is computed using the gradient of the loss function with respect to A :

$$\Delta A^{(t)} = \alpha \cdot \nabla_A \mathcal{L}(A, X)$$

where α is a learning rate. The gradient $\nabla_A \mathcal{L}(A, X)$ incorporates a Laplacian regularization term that ensures smooth feature transitions across edges and prevents abrupt changes in graph structure. This iterative refinement process ensures that the adjacency matrix $\bar{A}$ evolves towards a configuration that maximizes the model's robustness to adversarial perturbations while maintaining graph structure integrity.

Robustness to edge perturbations. Adversarial environments often involve deliberate perturbations, such as the addition or removal of edges, to mislead the model. Ensuring that RAGN is robust to such perturbations is essential for maintaining accuracy and reliability under potential adversarial attacks.

Proposition 4. *RAGN is robust to adversarial perturbations such as edge additions or deletions, ensuring that such modifications have minimal impact on attention scores and subsequent node classifications.*

Proof. When adversarial perturbations modify the graph structure by adding or deleting edges, the dynamic attention mechanism recalibrates the attention coefficients α_{uv} by adjusting the weights assigned to each edge. The recalibration rule is given by:

$$\alpha'_{uv} = \alpha_{uv} \times \sigma(-\beta \cdot d(u, v))$$

where $d(u, v)$ measures the deviation of the edge (u, v) from its expected weight, and σ is a sigmoid function scaled by β , a parameter that controls sensitivity to changes. The function $d(u, v)$ quantifies the deviation from normal edge patterns, and the sigmoid function ensures that edges that deviate significantly from the expected structure have their weights reduced, minimizing their impact on the attention scores. This recalibration ensures that the attention mechanism remains stable and resilient to adversarial modifications, as unexpected or deceptive edges are down-weighted, allowing the model to focus on the most relevant parts of the graph. In Section 3.3, we present empirical evidence that supports the theoretical enhancements introduced in RAGN. Our experimental results demonstrate that RAGN outperforms baseline models, such as GAT, in terms of both adversarial robustness and generalization to unseen attacks. This alignment between theoretical analysis and practical improvements underscores the efficacy of our approach in dynamic and adversarial environments.

3.5. The RAGN framework

To address the limitations of existing Graph Attention Networks in handling dynamic network environments and sophisticated adversarial threats, we propose a Robust Adaptive Graph Network (RAGN). This approach revises the traditional attention mechanisms in GATs to enhance the detection and defence against malicious network traffic. The

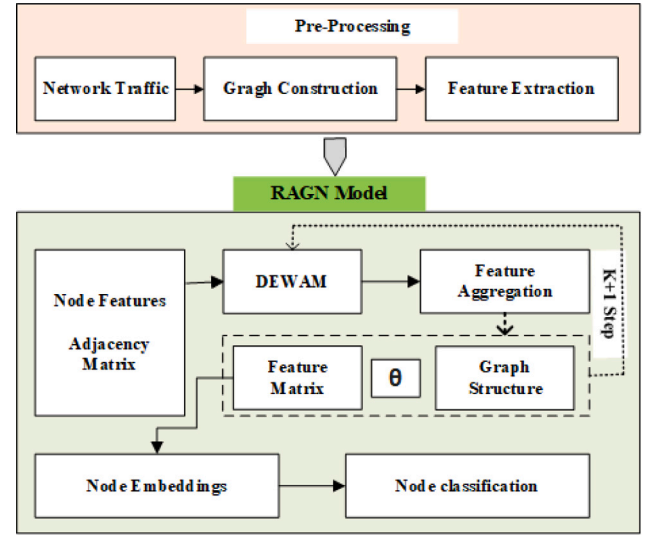


Fig. 2. RAGN-based malicious traffic detection roadmap.

overall workflow of the proposed RAGN method is illustrated in Fig. 2. RAGN introduces several key innovations. First, it incorporates an additional attention mechanism that considers edge reliability, assigning higher weights to positive edges and diminishing the influence of negative ones. Second, it employs an iterative optimization process to refine the graph structure, reducing E_2 and reinforcing E_1 . It further encourages connected nodes to have similar feature representations, aiding in distinguishing between benign and malicious entities.

Initially, standard preprocessing operations such as data segmentation, filtering, and normalization are performed on the raw network traffic data. This prepares the data for graph construction, where nodes represent network entities like IP addresses or devices, and edges represent interactions or traffic flows between these entities. Each node is initialized with features derived from the network data, encapsulating essential behavioral characteristics. In traditional GATs, attention scores are computed based solely on node features and the labels of neighboring nodes, as noted by Hu et al. [33]. The graph structure determines the choice of neighbors but does not influence the adjustment of attention scores. This limitation becomes significant in the presence of adversarial attacks, where attackers may add negative edges or remove positive edges to manipulate neighbor aggregation, thus degrading the model's performance. To overcome this challenge, RAGN introduces a dynamic adaptive attention mechanism that recalibrates attention scores in real-time, not only based on node features but also incorporating changes in the graph structure. By revising the aggregation function and inserting an extra attention score, RAGN effectively reduces the influence of adversarial edges. Intuitively, this mechanism assigns smaller attention scores to negative edges by utilizing prior information and continuous analysis to discern and mitigate the impact of malicious manipulations. This adjustment is crucial because the proper attention score is related not only to the structure A but also to the features X .

Furthermore, RAGN continuously adjusts both the graph structure and node features to generate this extra attention score, enhancing the model's robustness against adversarial attacks. This dynamic adaptation allows RAGN to remain sensitive to both the contextual relevance and reliability of node connections, improving its ability to withstand and respond to sophisticated adversarial strategies that aim to deceive the attention mechanism. After adjusting the attention mechanisms, RAGN employs a self-attention aggregation mechanism to synthesize insights across the network, producing a comprehensive representation that supports global analysis. This aggregated information is then fed into a softmax classification layer, which categorizes the network's condition

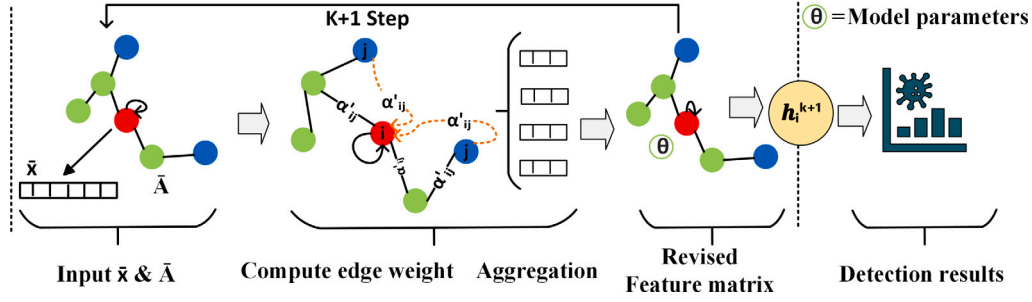


Fig. 3. Structure of the robust graph attention network framework.

Note: Schematic diagram of RAGN for a one-step iteration process. During one iteration, structure and feature are modified based on regularization first. Then, the modified structure is used to revise the attention score and the modified feature is used for nodes' information aggregation. Finally, the parameters of RAGN are updated based on the labeled nodes' loss function and the algorithm enters into the next iteration.

and identifies potential threats based on the overall network state. Each detected threat triggers a tailored response strategy, ranging from alerts to automated defensive actions, depending on the severity and nature of the identified threat. Feedback from these actions is utilized to continuously refine and update the model, enhancing its effectiveness and adaptability. Compared to existing methods, RAGN significantly improves upon traditional GATs by:

1. Extending the attention mechanism to adjust scores based on both node features and graph structure, addressing the limitations where the graph structure previously did not influence attention adjustments.
2. Introducing an extra attention scoring component specifically designed to mitigate the effects of adversarial edges by assigning them lower attention scores.
3. Continuously refining the graph structure and node features to generate adaptive attention scores that reduce the impact of adversarial manipulations.

Through these enhancements, RAGN provides a more robust and accurate system for detecting and defending against malicious network traffic, effectively addressing the challenges posed by dynamic and adversarial network environments. By integrating these innovative mechanisms, RAGN sets a new standard in network security technologies, offering a resilient and adaptable approach to safeguarding complex network infrastructures against evolving cyber threats. In Fig. 3, we show the schematic diagram of RAGN for a one-step iteration process. During one iteration, structure and feature are modified based on regularization first. Then, the modified structure is used to revise the attention score and the modified feature is used for nodes' information aggregation. Finally, the revised feature matrix, parameters, and graph structure are updated based on the labeled nodes' loss function and the algorithm enters into the next iteration.

3.6. Graph structure and node feature optimization

The original adjacency matrix A in GATs model the graph structure in a straightforward binary manner, representing the presence or absence of edges. The limitation of this representation is its inability to capture the strength or quality of connections, particularly critical in analyzing network traffic, where not all connections contribute equally to the network's functionality. This limitation motivates the need for a matrix $\bar{A}$ that reflects the existence of links and also their relevance based on node features and network dynamics.

To optimize the structure of the graph, we first introduces a revised adjacency matrix $\bar{A}$, which adjusts the original adjacency matrix A iteratively to enhance the robustness and reliability of GATs. In scenarios involving noise or adversarial attacks, the original graph structure represented by A may not accurately reflect the true underlying relationships among nodes, potentially degrading the model's performance.

The original adjacency matrix A is defined such that $A[i][j] = 1$ if there is an edge between nodes i and j , and $A[i][j] = 0$ otherwise. This binary setup, while straightforward, often fails to capture the nuanced interactions in complex data landscapes, particularly under adversarial conditions.

To address these challenges, we define an objective function that incorporates both structural fidelity and feature smoothness:

$$\mathcal{J}(\bar{A}) = \|A - \bar{A}\|_F^2 + \alpha \cdot \text{tr}(\bar{X}^T L \bar{X}) \quad (5)$$

Here, $\|A - \bar{A}\|_F$ measures the Frobenius norm, emphasizing minimal deviation from the original matrix A to maintain structural integrity. The term $\text{tr}(\bar{X}^T L \bar{X})$, where $L = D - \bar{A}$ is the Laplacian matrix and D is the diagonal degree matrix of $\bar{A}$, promotes feature homogeneity among connected nodes. The formulation of the objective function $\mathcal{J}(\bar{A})$ incorporates both the structural fidelity $\|A - \bar{A}\|_F^2$, ensuring minimal deviation from the observed data, and the feature-based regularization term $\alpha \cdot \text{tr}(\bar{X}^T L \bar{X})$. The balance between these terms determines the trade-off between adhering to the observed network structure and adapting to new insights derived from node features. The selection of the Frobenius norm and trace function is theoretically motivated by their properties in matrix analysis, providing a balance between simplicity and effectiveness in capturing matrix deviations and alignments.

The feature smoothness assumption underlying the Laplacian regularization suggests that connected nodes should exhibit similar features. This principle is essential in guiding the adaptive adjustments to $\bar{A}$, ensuring that connections in $\bar{A}$ are strengthened between nodes with similar features and weakened between those with dissimilar features. The use of the Laplacian matrix L in the objective function to promote feature smoothness across the graph leverages the Laplacian's properties in capturing the graph's structure. The quadratic form $\text{tr}(\bar{X}^T L \bar{X})$ minimizes the difference in features between directly connected nodes, effectively regularizing the learning process to favor smooth feature variations along the edges of the graph. This approach is underpinned by the theory of semi-supervised learning on graphs, where smoothness is a desired property for label propagation.

The iterative refinement of $\bar{A}$ employs a gradient descent algorithm, where at each step, $\bar{A}$ is adjusted to reduce the objective function $\mathcal{J}(\bar{A})$:

$$\bar{A} \leftarrow \bar{A} - \eta \nabla_{\bar{A}} \mathcal{J}(\bar{A}) \quad (6)$$

Here, $\nabla_{\bar{A}} \mathcal{J}(\bar{A})$ represents the gradient of the objective function with respect to $\bar{A}$, and η is the learning rate, determining the step size in the gradient descent. The iterative refinement process using gradient descent for updating $\bar{A}$ and $\bar{X}$ is rooted in optimization theory. The gradients $\nabla_{\bar{A}} \mathcal{J}(\bar{A})$ and $\nabla_{\bar{X}} \mathcal{J}(\bar{A})$ are calculated to guide the update steps to ensure each iteration moves towards a local minimum of the objective function. The use of gradient descent is justified by its effectiveness in handling large-scale data and its ability to converge to optimal solutions.

To ensure $\bar{A}$ remains a valid adjacency matrix throughout the optimization process, a projection step $\mathcal{P}_A(\cdot)$ is applied, which constrains each element of $\bar{A}$ to stay within $[0, 1]$. Additionally, regularization parameters like α are tuned to balance the trade-off between structural fidelity and feature smoothness. The projection steps $\mathcal{P}_A$ and $\mathcal{P}_X$ ensure that the revised matrices stay within feasible bounds, reflecting valid adjacency and feature representations. These projections are crucial for maintaining the physical interpretability and computational stability of the matrices.

In addition to adjusting the graph structure, RAGN iteratively revises the node features to suppress noise introduced by adversarial attacks. The objective is to ensure that the node features align with the revised graph structure, thereby improving the overall robustness. The feature matrix X is updated by minimizing the difference between the original features X and the revised features $\bar{X}$, while also enforcing feature smoothness using Laplacian regularization. The optimization problem for feature adjustment is formulated as:

$$\bar{X}^* = \arg \min_{\bar{X}} \|X - \bar{X}\|_2^2 + \gamma \text{tr}(\bar{X}^T L \bar{X}) + \lambda \mathcal{L}_{RAGN}(\theta, \bar{A}, \bar{X}, Y_p) \quad (7)$$

where γ is a non-negative parameter controlling the strength of the feature regularization, λ is a hyperparameter balancing the regularization and the loss term, and $\mathcal{L}_{RAGN}$ is the RAGN loss function that incorporates node classification accuracy on the labeled set Y_p . The first term ($\|X - \bar{X}\|_2^2$), minimizes the difference between the original features X and the updated features $\bar{X}$, ensuring the updates are not too drastic. The second term ($\text{tr}(\bar{X}^T L \bar{X})$), smoothens the features across connected nodes in the graph. It enforces the idea that nodes linked by edges should have similar features (a key assumption in graphs). ($\lambda \mathcal{L}_{RAGN}$) is the loss function that ensures the updated features help the model classify nodes (e.g., benign vs. malicious). The features are updated using gradient descent:

$$\bar{X} \leftarrow \mathcal{P}_X(\bar{X} - \eta_2 \nabla_{\bar{X}} (\|X - \bar{X}\|_2^2 + \gamma \text{tr}(\bar{X}^T L \bar{X}) + \lambda \mathcal{L}_{RAGN}(\theta, \bar{A}, \bar{X}, Y_p))) \quad (8)$$

where $\mathcal{P}_X(\cdot)$ projects the feature matrix onto the valid domain of features, and η_2 is the learning rate for the feature update. Together, these terms balance feature preservation, smoothness, and classification performance. The weights γ and λ control the importance of each aspect. This step updates node features to align them with the revised graph structure, making them robust and useful for classification. This two-step process of iteratively adjusting the graph structure and the feature matrix allows RAGN to progressively enhance the robustness of GAT against adversarial attacks. The next step involves dynamically generating attention scores to further mitigate the impact of adversarial edges. The next step in the RAGN framework involves generating dynamic attention scores that adjust based on the revised graph structure $\bar{A}$ and features $\bar{X}$. This step is critical to mitigating the impact of adversarial edges by assigning appropriate weights to both positive and negative edges during the node aggregation process.

The revision of A to $\bar{A}$ is based on the hypothesis that nodes with similar features should have stronger connectivity if they contribute positively to the task (e.g., classification, clustering). This assumption is rooted in the homophily principle prevalent in scenarios where similar nodes tend to be more connected. The theoretical justification for enhancing connections based on feature similarity is further supported by studies in spectral graph theory, which suggest that closely connected nodes in a graph's Laplacian eigenspace tend to share similar features and labels.

3.7. Attention mechanism

In traditional GATs, the attention mechanism calculates attention coefficients α_{uv} between neighboring nodes u and v based solely on their features. The attention mechanism in RAGN is designed to address critical challenges in this traditional graph representation learning, particularly in adversarial and dynamic environments. For instance, GNNs

aggregate node information using static weights, treating all edges equally during message passing. This approach is limited because it fails to account for variations in the importance of edges, leaving the model vulnerable to adversarial attacks that manipulate graph structure or node features. Adversarial perturbations, such as the addition, deletion, or modification of edges, distort the graph's underlying patterns, misleading the model and degrading its predictive accuracy. To overcome these limitations, we introduce an attention mechanism to dynamically assign weights to edges, prioritizing meaningful relationships while suppressing the influence of irrelevant or noisy connections.

The core of the attention mechanism is the computation of edge-specific attention scores, which reflect the relevance of each edge to the task at hand. For an edge (i, j) connecting nodes i and j , the attention score α_{ij} is calculated as:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}, \quad (9)$$

where e_{ij} is the unnormalized attention coefficient, and $\mathcal{N}(i)$ denotes the set of neighbors of node i . The softmax function ensures that the attention scores are normalized, making them interpretable and ensuring that their sum over all neighbors is equal to one.

The unnormalized coefficient e_{ij} is computed using a learnable mechanism that captures both feature similarity and task-specific relationships. Specifically:

$$e_{ij} = \text{LeakyReLU}(a^T [W F_i \parallel W F_j]), \quad (10)$$

where F_i and F_j are the feature vectors of nodes i and j , $W \in \mathbb{R}^{d' \times d}$ is a learnable linear transformation matrix, $a \in \mathbb{R}^{2d'}$ is a learnable vector that projects concatenated node embeddings into the attention space, and $\parallel$ denotes concatenation. The LeakyReLU activation introduces non-linearity, enabling the attention mechanism to model complex interactions between node features.

The computed attention scores are used to refine the adjacency matrix. The revised adjacency matrix $\bar{A}$ is updated iteratively based on the following gradient descent rule:

$$\bar{A}_{ij}^{(k+1)} = \bar{A}_{ij}^{(k)} - \eta \frac{\partial \mathcal{L}}{\partial \bar{A}_{ij}}, \quad (11)$$

where k represents the iteration step, η is the learning rate, and $\frac{\partial \mathcal{L}}{\partial \bar{A}_{ij}}$ is the gradient of the loss function $\mathcal{L}$ with respect to $\bar{A}_{ij}$. This iterative process ensures that the adjacency matrix dynamically adapts to capture meaningful relationships while mitigating the effects of adversarial perturbations.

The dynamic attention mechanism employed in our model is designed to enhance adaptability across various network traffic conditions. Traditional attention mechanisms assign pre-defined weights to different input features, limiting their effectiveness when dealing with dynamic and evolving network behaviors. In contrast, our approach utilizes a self-adaptive weighting strategy that enables the model to focus on the most relevant traffic characteristics at any given time. This adaptability is particularly beneficial in network environments where traffic patterns fluctuate due to varying application demands, congestion levels, or external factors such as network attacks or hardware failures. The mechanism dynamically adjusts its attention weights based on both spatial and temporal dependencies, allowing it to generalize effectively across different traffic scenarios. Whether the network experiences stable, bursty, or periodic traffic, the model can reallocate its focus to capture critical trends and correlations, ensuring robust performance across diverse conditions. It does not rely on specific traffic types or predefined rules. Instead, it learns the underlying structure of network flows from raw input data, making it well-suited for heterogeneous network environments.

In order to achieve this, the attention mechanism in RAGN is grounded in two theoretical principles: feature smoothness and structural consistency. Feature smoothness assumes that nodes connected by meaningful edges have similar feature representations. This principle

is enforced through a regularization term that penalizes differences in features between connected nodes:

$$\Psi(\bar{A}, F) = \frac{1}{2} \sum_{i,j} \bar{A}_{ij} \|F_i - F_j\|^2, \quad (12)$$

where F_i and F_j are the feature vectors of nodes i and j . Structural consistency ensures that the revised adjacency matrix $\bar{A}$ aligns with the original graph structure, preventing excessive deviation. This is achieved by incorporating the graph Laplacian $L = D - A$, where D is the degree matrix, into a structural regularization term:

$$\Phi(\bar{A}) = \text{Tr}(\bar{A}^T L \bar{A}). \quad (13)$$

The overall optimization objective combines these regularization terms with the task-specific loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_1 \Phi(\bar{A}) + \lambda_2 \Psi(\bar{A}, F), \quad (14)$$

where λ_1 and λ_2 are trade-off parameters balancing the contributions of structural consistency and feature smoothness. In adversarial settings, the attention mechanism significantly enhances robustness. By assigning lower weights to perturbed edges, it reduces the sensitivity of the graph embeddings to adversarial perturbations. This robustness is theoretically supported by the bound on the Frobenius norm of the perturbed adjacency matrix:

$$\|\Delta A\|_F \leq \epsilon, \quad (15)$$

where ϵ represents the adversarial perturbation budget. By reweighting edges dynamically, the mechanism ensures that ΔA has minimal impact on the model's performance. Through these properties, the attention mechanism in RAGN provides a robust, and adaptive framework for learning from graph-structured data, addressing the challenges of adversarial perturbations, dynamic graphs, and heterogeneous relationships. The mechanism's ability to selectively prioritize meaningful edges while mitigating noise makes it a critical component of the RAGN model.

In RAGN, the attention score is enhanced by incorporating the revised graph structure $\bar{A}$ and generating an additional dynamic attention score $\tilde{\alpha}_{uv}$ for each edge. The revised attention score $\tilde{\alpha}_{uv}$ is computed as:

$$\tilde{\alpha}_{uv} = \bar{a}_{uv} \cdot \alpha_{uv} \quad (16)$$

Here, $\bar{a}_{uv}$ is the revised edge weight obtained from the optimization of the adjacency matrix $\bar{A}$, which reflects the structural adjustments made to mitigate adversarial attacks. The goal is to down-weight the influence of adversarial edges and amplify the importance of positive edges. Specifically, $\bar{a}_{uv}$ is derived from the structure optimization in Step 1 and is used to adjust the original attention scores α_{uv} . The final attention score used for aggregation is then:

$$\hat{\alpha}_{uv} = \frac{\tilde{\alpha}_{uv}}{\sum_{k \in \mathcal{N}(v)} \tilde{\alpha}_{vk}} \quad (17)$$

This dynamic attention mechanism ensures that edges identified as adversarial (i.e., negative edges) receive lower attention scores, while beneficial edges (positive edges) receive higher attention scores. By iterating this process, RAGN progressively increases the ratio of attention scores for real edges to adversarial edges, thus improving the overall robustness of the model. The revised aggregation function for node v in layer k of RAGN becomes:

$$\mathbf{x}_v^{(k)} = \sigma \left(\sum_{u \in \mathcal{N}(v)} \hat{\alpha}_{uv} W^{(k)} \mathbf{x}_u^{(k-1)} \right) \quad (18)$$

where $\hat{\alpha}_{uv}$ is the adjusted attention score, $W^{(k)}$ is the weight matrix for layer k , and $\mathbf{x}_u^{(k-1)}$ is the feature vector of node u from the previous layer. This dynamic attention score generation step completes the RAGN process, allowing the network to adapt to adversarial attacks by revising both the structure and the attention mechanism, leading to more robust node embeddings.

3.8. Optimization process of RAGN

The optimization of RAGN involves iteratively updating the graph structure, node features, and model parameters to ensure robust performance under adversarial attacks. The overall objective function combines the regularization terms for both graph structure and features, along with the classification loss on the labeled nodes. The optimization process follows an alternating minimization approach, where the graph structure and node features are updated first, followed by the model parameters. The optimization process of RAGN is driven by minimizing the following objective function:

$$\mathcal{L}_{\text{total}}(\bar{A}, \bar{X}, \theta) = \mathcal{L}_{\text{re}}(\bar{A}, \bar{X}) + \lambda \mathcal{L}_{\text{RAGN}}(\theta, \bar{A}, \bar{X}, Y_p) \quad (19)$$

This equation combines two objectives to optimize the RAGN model where $\mathcal{L}_{\text{re}}(\bar{A}, \bar{X})$ is the regularization loss that enforces smoothness in the graph structure and feature matrix. $\mathcal{L}_{\text{RAGN}}(\theta, \bar{A}, \bar{X}, Y_p)$ is the classification loss for the labeled nodes Y_p , using the RAGN parameters θ . λ is a hyperparameter balancing the regularization and classification loss. The term $\mathcal{L}_{\text{re}}(\bar{A}, \bar{X})$, regularizes the graph structure ($\bar{A}$) and the node features ($\bar{X}$), ensuring they are smooth and resistant to adversarial attacks. It penalizes large changes to the graph and features unless they improve robustness. The $\lambda \mathcal{L}_{\text{RAGN}}$ term focuses on improving classification accuracy. It trains the model parameters θ to correctly classify labeled nodes (Y_p) based on the updated graph structure and features. The hyperparameter λ balances the trade-off between robustness and classification accuracy. This ensures that RAGN learns to simultaneously protect the graph from adversarial attacks and perform accurate node classification. The regularization loss $\mathcal{L}_{\text{re}}$ is defined as:

$$\mathcal{L}_{\text{re}}(\bar{A}, \bar{X}) = \|A - \bar{A}\|_2^2 + \beta \|X - \bar{X}\|_2^2 + \alpha \text{tr}(\bar{X}^T L \bar{X}) \quad (20)$$

where $\|A - \bar{A}\|_2^2$ penalizes the difference between the original adjacency matrix A and the revised matrix $\bar{A}$. $\|X - \bar{X}\|_2^2$ penalizes the difference between the original feature matrix X and the revised features $\bar{X}$. $\text{tr}(\bar{X}^T L \bar{X})$ is the Laplacian regularization term that enforces smoothness in the revised feature matrix $\bar{X}$. α and β are hyperparameters controlling the influence of these terms. The optimization procedure alternates between updating the graph structure $\bar{A}$, the feature matrix $\bar{X}$, and the RAGN parameters θ . The first step is to update the adjacency matrix $\bar{A}$, keeping the feature matrix $\bar{X}$ and the RAGN parameters θ fixed. This is achieved by minimizing the regularization loss $\mathcal{L}_{\text{re}}$ with respect to $\bar{A}$:

$$\bar{A}^* = \arg \min_{\bar{A}} \|A - \bar{A}\|_2^2 + \alpha \text{tr}(\bar{X}^T L \bar{X}) \quad (21)$$

The update is performed using projected gradient descent, ensuring that $\bar{A}$ remains a valid adjacency matrix (i.e., all elements between 0 and 1):

$$\bar{A} \leftarrow \mathcal{P}_{\mathcal{A}}(\bar{A} - \eta_1 \nabla_{\bar{A}}(\|A - \bar{A}\|_2^2 + \alpha \text{tr}(\bar{X}^T L \bar{X}))) \quad (22)$$

where η_1 is the learning rate for the graph structure update, and $\mathcal{P}_{\mathcal{A}}(\cdot)$ is the projection operator ensuring $\bar{A} \in [0, 1]$. Next, the feature matrix $\bar{X}$ is updated while holding the graph structure $\bar{A}$ and RAGN parameters θ fixed. The optimization problem for the feature matrix is:

$$\bar{X}^* = \arg \min_{\bar{X}} \|X - \bar{X}\|_2^2 + \gamma \text{tr}(\bar{X}^T L \bar{X}) + \lambda \mathcal{L}_{\text{RAGN}}(\theta, \bar{A}, \bar{X}, Y_p) \quad (23)$$

The feature matrix is updated using gradient descent:

$$\bar{X} \leftarrow \mathcal{P}_{\mathcal{X}}(\bar{X} - \eta_2 \nabla_{\bar{X}}(\|X - \bar{X}\|_2^2 + \gamma \text{tr}(\bar{X}^T L \bar{X}) + \lambda \mathcal{L}_{\text{RAGN}}(\theta, \bar{A}, \bar{X}, Y_p))) \quad (24)$$

where η_2 is the learning rate for the feature update, and $\mathcal{P}_{\mathcal{X}}(\cdot)$ is the projection operator that ensures $\bar{X}$ remains a valid feature matrix. Finally, the RAGN parameters θ are updated, keeping the revised graph structure $\bar{A}$ and the revised feature matrix $\bar{X}$ fixed. The RAGN classification loss $\mathcal{L}_{\text{RAGN}}$ is minimized to optimize the model parameters:

$$\theta^* = \arg \min_{\theta} \mathcal{L}_{\text{RAGN}}(\theta, \bar{A}, \bar{X}, Y_p) \quad (25)$$

The parameter update is performed using gradient descent:

$$\theta \leftarrow \theta - \eta_3 \nabla_{\theta} \mathcal{L}_{\text{RAGN}}(\theta, \bar{A}, \bar{X}, Y_p) \quad (26)$$

where η_3 is the learning rate for updating the RAGN parameters. The optimization process alternates between updating $\bar{A}$, $\bar{X}$, and θ for a fixed number of iterations or until convergence. The alternating updates ensure that the graph structure and feature matrix progressively improve, making the model more robust to adversarial attacks. At each iteration, the graph structure is refined to emphasize positive edges and reduce the influence of adversarial edges. The feature matrix is adjusted to align with the revised structure, and the RAGN parameters are optimized to improve node classification accuracy under adversarial conditions. The convergence of the process is guaranteed as each step involves minimizing a convex objective with respect to the current variable (graph structure, features, or parameters), leading to stable updates at each iteration.

Algorithm 1 The RAGN Algorithm

```

1: Input: Graph  $G = (V, A, X)$ , Partially labeled nodes  $V_p$  with labels  $Y_p$ 
2: Parameters: Learning rates  $\eta_1, \eta_2, \eta_3$ ; Regularization parameters  $\alpha, \beta, \gamma, \lambda$ ; Max iterations  $T_1, T_2$ 
3: Output: Optimized parameters  $\theta^*$ , Revised adjacency matrix  $\bar{A}$ , Revised feature matrix  $\bar{X}$ 
4: Initialize  $\bar{A} \leftarrow A$ ,  $\bar{X} \leftarrow X$ , randomly initialize RAGN parameters  $\theta$ 
5: for  $i = 1$  to  $T_1$  do
6:   Graph Structure Update:
7:   Compute gradient:  $\nabla_{\bar{A}} = \nabla_{\bar{A}} (\|\bar{A} - A\|_2^2 + \alpha \text{tr}(\bar{X}^T L \bar{X}))$ 
8:   Update adjacency matrix:  $\bar{A} \leftarrow \mathcal{P}_{\bar{A}} (\bar{A} - \eta_1 \nabla_{\bar{A}})$ 
9:   Feature Matrix Update based on gradient:
10:   $\nabla_{\bar{X}} = \nabla_{\bar{X}} (\|X - \bar{X}\|_2^2 + \gamma \text{tr}(\bar{X}^T L \bar{X}) + \lambda \mathcal{L}_{\text{RAGN}}(\theta, \bar{A}, \bar{X}, Y_p))$ 
11:  Update feature matrix:  $\bar{X} \leftarrow \mathcal{P}_{\bar{X}} (\bar{X} - \eta_2 \nabla_{\bar{X}})$ 
12:  for  $j = 1$  to  $T_2$  do
13:    RAGN Parameter Update:
14:    Compute gradient:  $\nabla_{\theta} = \nabla_{\theta} \mathcal{L}_{\text{RAGN}}(\theta, \bar{A}, \bar{X}, Y_p)$ 
15:    Update parameters:  $\theta \leftarrow \theta - \eta_3 \nabla_{\theta}$ 
16:  end for
17: end for
18: Return Optimized  $\theta^*$ ,  $\bar{A}$ , and  $\bar{X}$ 

```

3.9. Training process of RAGN

The training process of RAGN for detecting unknown malicious network traffic involves an iterative optimization strategy that alternates between refining the graph structure, updating the node features, and optimizing the model parameters. This process aims to strengthen the model's ability to distinguish between normal and malicious network traffic, even in the presence of adversarial efforts to obfuscate malicious behavior. In the context of network traffic detection, the network is represented as a graph where nodes correspond to traffic entities (such as IP addresses, network flows, or devices), and edges represent relationships or interactions between them. The adjacency matrix encodes these connections, while the feature matrix captures relevant characteristics of the traffic, such as packet size, source and destination addresses, or timing information. The first step in the training process is to revise the graph's adjacency matrix. This is essential for improving the model's resilience to adversarial attacks, where malicious entities may attempt to manipulate the network structure to disguise their behavior. RAGN adjusts the adjacency matrix by encouraging stronger connections between nodes that exhibit similar traffic patterns (such as normal traffic nodes) and weakening connections that may have been adversarially manipulated to obscure the nature of the malicious nodes.

This adjustment ensures that the connections in the graph better reflect the true relationships between entities in the network, thereby

enhancing the model's ability to detect anomalous or malicious activity. The revised adjacency matrix highlights the reliable connections while reducing the influence of misleading, adversarial edges. Once the adjacency matrix has been revised, the next step is to update the feature matrix. The goal of this step is to ensure that the node features accurately reflect the true behavior of the network traffic while minimizing the effects of adversarial noise. In the case of network traffic, these features may include packet-related data, IP addresses, communication frequency, or other attributes relevant to distinguishing between benign and malicious entities. The feature refinement process adjusts the node features to align with the newly revised graph structure, ensuring that traffic nodes with similar behavior remain closely related in the feature space, while nodes representing malicious traffic are more distinctly separated. This step is critical for suppressing any noise or perturbations introduced by adversarial actors, enabling the model to more effectively capture meaningful patterns in the data. After refining the graph structure and features, the model parameters are updated to optimize its classification performance. The RAGN model learns to classify nodes (network entities) as either benign or malicious based on the revised adjacency matrix and updated feature representations. This involves training the model to recognize patterns of normal and malicious behavior within the network traffic graph and to generalize to unknown malicious activity.

During this step, the model adjusts its parameters to minimize classification errors on the labeled training data, where certain traffic entities are known to be either benign or malicious. As the model parameters are optimized, the model becomes increasingly accurate at detecting malicious traffic, even when the malicious nodes or interactions have been adversarially manipulated to evade detection. The training process alternates between these three steps – graph structure adjustment, feature refinement, and parameter optimization – over several iterations. With each iteration, the graph structure, node features, and model parameters are refined to improve the model's robustness to adversarial attacks and enhance its classification accuracy. The process continues until a convergence criterion is met, such as when the changes in the graph, features, and parameters between iterations become negligible, or when a maximum number of iterations is reached. By iteratively refining both the structure of the network and the features of the traffic entities, RAGN progressively enhances its ability to detect unknown malicious traffic. The final model is robust to sophisticated adversarial attacks, effectively distinguishing between normal and malicious network traffic even when malicious actors attempt to manipulate the graph structure or obfuscate their traffic behavior. This makes RAGN a powerful tool for network security, enabling the detection of unknown and evolving threats in network traffic.

3.10. Overview of proposed adversarial attack method

To evaluate the robustness of the proposed RAGN model and baseline state-of-the-art models, we introduce Semantic-Preserving Adversarial Node Injection (SPAN), a novel adversarial attack method specifically tailored for dynamic graph neural networks. Unlike general-purpose attacks, SPAN ensures that injected perturbations align with the semantic integrity of network traffic, preserving natural communication behaviors while effectively disrupting graph structures. This approach makes the attack not only realistic and domain-relevant but also effective in exposing model vulnerabilities. Network traffic graphs are dynamic, evolving over time as $G = \{G_0, G_1, \dots, G_T\}$, where each snapshot $G_i = (V_i, A_i, F_i)$ consists of nodes V_i (e.g., endpoints or devices), edges A_i (e.g., communication links), and features F_i (e.g., traffic statistics). SPAN operates with the following key objectives:

- **Maximizing Model Disruption:** SPAN targets vulnerable nodes and edges to degrade the performance of tasks like node classification or link prediction.

- Preserving Imperceptibility through semantic alignment: The attack ensures that injected perturbations remain undetected by anomaly detection systems through semantic alignment.
- Exploiting Temporal and Structural Fragility: By leveraging both temporal traffic patterns and structural vulnerabilities, SPAN strategically injects perturbations that align with real-world dynamics.
- Adhering to Budget Constraints: SPAN operates within predefined limits, such as the number of injected nodes (b), degree budget (d), and feature range (ΔF), ensuring computational efficiency and realism.

The attack involves injecting a set of malicious nodes V_t^I with features F_t^I into G_t , connecting them to existing nodes V_t via edges E_t^I . The resulting perturbed graph G_t' is represented as:

$$A_t' = \begin{pmatrix} A_t & E_t^I \\ (E_t^I)^T & A_t^I \end{pmatrix}, \quad F_t' = \begin{pmatrix} F_t \\ F_t^I \end{pmatrix}. \quad (27)$$

Here, A_t^I and E_t^I represent the internal connections between malicious nodes and their connections to the original nodes, respectively. To maintain semantic coherence, SPAN ensures that the injected nodes simulate plausible traffic behaviors, such as spoofed flows, fake connections, or modified communication patterns, without violating the natural structure of network traffic graphs. With these features, SPAN provides a rigorous method for evaluating the robustness of graph-based models in realistic network traffic scenarios. By targeting both the structural and temporal aspects of dynamic graphs while preserving semantic integrity, SPAN ensures effective and imperceptible perturbations that challenge the resilience of modern graph neural networks.

3.10.1. Semantic-preserving adversarial node injection

To exploit the vulnerability of graph neural networks used in network traffic classification, the proposed SPAN puts forward a stepwise injection strategy, progressively injecting nodes in small batches to iteratively refine their connections and features. Vulnerable nodes are selected based on both structural fragility quantified by a node's importance to graph connectivity and semantic relevance, such as critical roles in traffic flows. The process comprises three key steps:

1. Selecting the optimal time moments for injection based on temporal traffic patterns.
2. Identifying structurally fragile edges and nodes using traffic dynamics and vulnerability scores.
3. Optimizing malicious node features with smoothed adversarial feature optimization to balance attack impact and stealth.

In this method, the node aggregation process for a given node v at time t in the graph neural network is defined as:

$$h_v^{k,t} = \phi \left(h_v^{k-1,t}, f \left(\{ h_u^{k-1,t} \mid u \in A_t(v) \} \right) \right), \quad (28)$$

$$\psi \left(h_v^{k-1}, \{ h_u^{k-1} \mid u \in A_{t-1}(v) \} \right)$$

where $h_v^{k,t}$ is the hidden representation of node v at layer k and time t , $\phi(\cdot)$ is the update function for node embeddings, and $\psi(\cdot)$ aggregates historical information. The set $A_t(v)$ includes the neighbors of v directly or indirectly connected within a specific radius. Assuming the attack occurs at time t , the goal is to perturb the embedding $h_v^{k,t}$ by modifying the neighbors $A_t(v)$ via injected nodes. Injected nodes $v' \in V_t^I$ are initialized with features $h_{v'}^{k-1,t} = 0$, ensuring realistic integration into the graph. The impact of the injection on the embedding $h_v^{k,t}$ can be expressed as:

$$\Delta h_v^{k,t} = h_v^{k,t,m'} - h_v^{k,t} = \sum_{m=0}^n p_m k_m h_u^{k-1,t} + o \left(\sum_m \Delta f_m \right), \quad (29)$$

where p_m represents the propagation coefficients, and k_m reflects the impact weight from the neighborhood m .

The graph's dynamic structure is analyzed across time intervals T , with the moment of injection $D = \{G_{T_{i-2}}, G_{T_{i-1}}, \dots\}$ determined by maximizing structural vulnerability and traffic relevance. The temporal traffic intensity $g(t)$ serves as the proportional weight:

$$g(t) = \frac{\text{traffic}(t)}{\max_t \text{traffic}(t)}. \quad (30)$$

Injected nodes are strategically connected to nodes in V_t with high $g(t)$ and fragility scores δ_v . Injected edges E_t^I are designed to target nodes within the m -hop neighborhood $A_t^m(v)$ while maintaining realistic communication behaviors. The overall change in the embedding function due to injected edges is expressed as:

$$\Delta f_m = \sum_{u \in A_t^m(v)} w_{u,m} h_u^{k-1,t}, \quad (31)$$

where $w_{u,m}$ represents the weights of the affected neighborhood nodes. By focusing on the temporal traffic patterns and structural fragility of communication nodes, this method ensures that injected perturbations align with plausible network behaviors, addressing both structural disruption and semantic integrity.

During the edge selection phase, the features of malicious nodes have not been fully determined. Therefore, to maximize the perturbation impact, we focus on maximizing Δf_1 , which is expressed as:

$$\Delta f_1 = \sum_{u \in A_t^1(v)} \Delta w_{u,1} h_u^{0,t} + \sum_{u \in A_t^1(v'), A_t^1(v)} w_{u,1} h_u^{0,t'}, \quad (32)$$

where $\Delta w_{u,1}$ and $w_{u,1}$ denote the respective weights in the perturbed and original graphs, and $h_u^{0,t}$ refers to the hidden representation of node u before the attack. This strategy ensures that the injected nodes target structurally significant connections while maintaining the semantic coherence of the graph.

To determine the most effective edges for connection, we rely on the structural fragility score of the target nodes. For graph convolutional networks (GCNs) and mean-pooling-based graph neural networks, the weights $w_{u,1}$ for connections are computed as:

$$w_{u,1} = \frac{1}{\deg(u) \deg(v)}, \quad \Delta w_{u,1} = \frac{1}{\deg(v)}, \quad (33)$$

where $\deg(u)$ and $\deg(v)$ are the degrees of the original and injected nodes, respectively. These weights capture the structural vulnerability of the graph, ensuring that perturbations focus on nodes critical to the graph's overall connectivity and traffic flow.

The structural fragility score δ_v of a node v is defined as:

$$\delta_v = k_1 \cdot \frac{1}{\deg(u) \deg(v)} + k_2 \cdot \frac{1}{\deg(u)}, \quad (34)$$

where k_1 and k_2 are hyperparameters that balance the influence of global and local properties. Nodes with higher δ_v scores are more vulnerable to graph injection attacks because perturbations involving these nodes significantly impact Δf_1 .

To limit the number of edges connected to each injected node and ensure the attack remains realistic, a degree budget σ_t is introduced for the injected nodes such that:

$$\sigma_t = d \cdot N_I, \quad (35)$$

where N_I is the total number of injected nodes, and d is the degree limit for each node. The target node set P is then defined as:

$$P = \{\text{rank}(\delta_v) \leq \sigma_t, \forall v \in G_t, T_{i-2}, T_{i-1}, \dots\}. \quad (36)$$

Here, P includes all nodes v with δ_v -value rankings less than the degree budget σ_t for the selected graph snapshot G_t . By ensuring that the injected nodes are connected to structurally fragile and traffic-relevant nodes, this strategy guarantees the realism of the perturbations in network traffic graphs. Furthermore, the inclusion of degree and temporal constraints ensures that the attack adheres to real-world communication dynamics, making it suitable for evaluating the robustness of graph neural networks under realistic adversarial scenarios.

3.10.2. Adversarial feature generation

After selecting structurally fragile nodes for adversarial connections, the next step is to assign meaningful and realistic features to the injected malicious nodes. Since the features of malicious nodes are initially set to zero, it is essential to optimize them in a way that maximizes the attack's impact on model predictions while maintaining imperceptibility. This is particularly important for features such as packet size, protocol type, and flow duration to appear plausible to evade detection. To achieve this, we propose a smoothed adversarial feature optimization method. This method generates features F_t^I for malicious nodes such that they align with realistic traffic patterns while maximizing the perturbation effect on the model. Specifically, the features are constrained within a valid range to avoid triggering anomaly detection mechanisms in the model. To limit the features of malicious nodes and prevent detection, we use a random clipping function:

$$\text{Random}(x) = \begin{cases} \min, & x < \min \\ x, & \min \leq x \leq \max \\ \max, & x > \max, \end{cases} \quad (37)$$

where $\min$ and $\max$ are the lower and upper bounds of the feature range, respectively. This ensures that the features of malicious nodes stay within a domain-specific valid range, such as typical values observed in network traffic (e.g., realistic packet sizes or flow frequencies).

However, the random clipping function can lead to a zero gradient problem, where feature values are clipped at their boundaries, preventing further optimization. This can cause the features to stagnate and reduce the effectiveness of the attack. To address this limitation, we introduce a smooth and continuous feature optimization function. The proposed smoothed optimization function, $\text{Op}_{\text{feature}}(x)$, avoids the zero gradient issue by mapping features onto a continuous range:

$$\text{Op}_{\text{feature}}(x) = \frac{\max(1 + \sin(x)) + \min(1 - \sin(x))}{2}. \quad (38)$$

This function ensures that the features of injected malicious nodes are smoothly updated during optimization, preventing them from becoming stuck at the boundaries defined by $\text{Random}(x)$. By using $\text{Op}_{\text{feature}}(x)$, we maintain a balance between imperceptibility and the ability to disrupt model predictions. The optimization of malicious node features aims to maximize the model's prediction error while adhering to the constraints of imperceptibility. The objective function is defined as:

$$\min_{F_t^I} \|M(G_t') - M(G_t)\| + \lambda \|F_t^I - \mu(F_t)\|, \quad (39)$$

where:

- $M(G_t)$ and $M(G_t')$ represent the model's predictions on the original and perturbed graphs, respectively.
- $\mu(F_t)$ is the mean feature vector of the original nodes in the graph, ensuring that the malicious features are consistent with normal network traffic patterns.
- λ is a hyperparameter that balances the trade-off between maximizing the attack's impact and maintaining imperceptibility.

By using $\text{Op}_{\text{feature}}(x)$, the optimization process ensures that malicious nodes blend seamlessly into the graph while effectively perturbing the model's predictions. This approach prevents detection by anomaly-based defenses and guarantees that the perturbations respect the semantic integrity of network traffic.

3.10.3. Overall attack process of SPAN

The proposed SPAN attack is designed as a stepwise process, particularly suitable for network traffic graphs. The attack process of SPAN is illustrated in Fig. 4. The method leverages a batch injection strategy, where a small number of malicious nodes are added to the graph in each round, and their features are iteratively optimized to maintain

semantic integrity and maximize the attack's impact. This iterative approach ensures the attack respects network-specific constraints, such as communication behaviors and temporal traffic dynamics, while remaining imperceptible to anomaly detection mechanisms. SPAN begins by selecting target nodes based on their structural fragility and traffic relevance. Using the total number of malicious nodes and the degree budget constraints, the method determines the number of target nodes to attack. For each graph snapshot, the structural fragility score of all nodes is computed using Eq. (34), and the most vulnerable nodes are identified for connection with malicious nodes. The injected nodes are then connected to these targets, and their features are optimized using Eq. (38) to ensure alignment with realistic network traffic patterns. The stepwise injection process repeats until the attack budget (number of malicious nodes b) is exhausted. After each round of injection, the attack budget is updated, and the algorithm proceeds to the next round. This iterative refinement ensures that the injected nodes blend seamlessly into the dynamic network traffic graph and maximize the degradation of model performance.

Algorithm 2 The process of the SPAN attack method

Require:

- 1: Clean graph $G = \{A_{-n}, A_{-n+1}, \dots, A_T\}$, $X = \{F_{-n}, F_{-n+1}, \dots, F_T\}$
- 2: Parameters:
 - b : Budget on the number of injected nodes
 - d : Degree budget of each injected node
 - ΔF : Constraint on the range of node features

Ensure: Attacked graph $G' = \{A'_{-n}, \dots, A'_T\}$, $X' = \{F'_{-n}, \dots, F'_T\}$

- 3: Initialize $S' \leftarrow S + S_I$, $F' \leftarrow F + F_I$
- 4: Select target time snapshots $D = \{G_{T-2}, G_{T-1}, \dots, G_{T-q}, \dots\}$
- 5: **for** $G_t \in D$ **do**
- 6: Initialize malicious node set $I \in \mathbb{R}^{0 \times N_I}$, connection matrix $E_t^I \in \mathbb{R}^{0 \times d}$, and feature matrix $F_t^I \in \mathbb{R}^{0 \times \Delta F}$
- 7: **while** $b > 0$ **do**
- 8: **for** $v \in G_t$ **do**
- 9: Compute defective fragility score δ_v using Eq. (12)
- 10: **end for**
- 11: Select the number of malicious nodes $b_{\text{seq}} \propto \frac{g(t)}{|D|}$
- 12: Connect b_{seq} malicious nodes to the highest fragility target nodes
- 13: Initialize and optimize malicious node features F_t^I using Eq. (15)
- 14: Update the attack budget $b \leftarrow b - b_{\text{seq}}$
- 15: **end while**
- 16: Update A'_t and F'_t using Eq. (1) and Eq. (2)
- 17: **end for**
- 18: **return** $G' = \{A'_0, F'_0\}, \dots, \{A'_T, F'_T\}$

4. Experimental setup and evaluation

4.1. Implementation details

In our experiments, we employ a two-layer GAT model, applying a dropout rate of 0.6 to the input of both layers. The model is trained using Stochastic Gradient Descent (SGD) with a learning rate of 0.01, applied to both the feature matrix and the adjacency matrix. This learning rate ensures that updates are gradual, avoiding rapid convergence that could lead to suboptimal solutions. The dropout rate helps prevent overfitting, especially when training on smaller datasets, while also maintaining stability in the model's performance. The graph structure adjustment strength (α), with values between 0.01 and 1.0; the feature smoothing coefficient (β), tested from 0.01 to 1.0 to assess its effect on feature smoothness across nodes; and the adversarial loss coefficient (λ), which was varied between 0.01 and 1.0

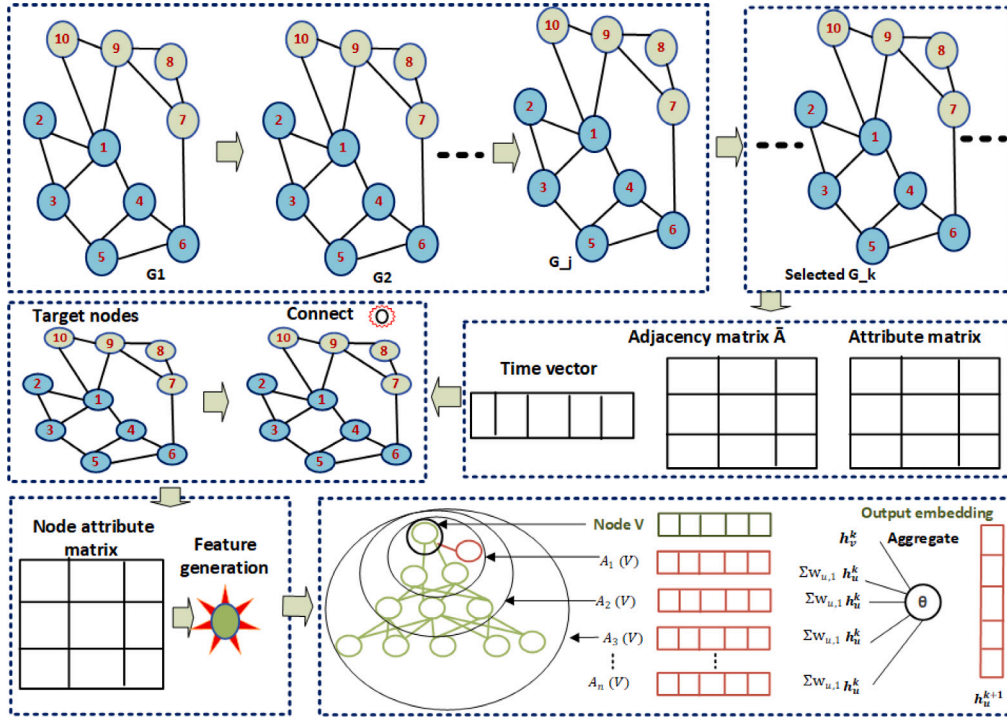


Fig. 4. Dynamic graph attack process by SPAN.

to control the trade-off between classification accuracy and adversarial robustness. Once the optimal parameters were selected based on validation performance, the final evaluation was conducted on the test set. Additionally, robustness was tested by introducing both targeted and untargeted adversarial attacks, with perturbation rates ranging from 0% to 25%, enabling us to measure how effectively each model can resist adversarial manipulations in the test data.

In the graph construction process, edge weights were determined based on the interactions between network entities. Specifically, an edge between two nodes (e.g., IP addresses) was established if there was at least one bidirectional traffic flow between them. The weight w_{ij} of an edge connecting nodes i and j was calculated as a function of the total bytes exchanged, normalized by the maximum traffic volume observed across all edges. This ensured that edge weights reflected the relative strength of interactions. For temporal graphs, edge weights were dynamically updated within a time window Δt , incorporating the frequency of interactions to capture evolving network behaviors. In adversarial scenarios, such as those involving malicious node injections, edge weights for injected nodes were constrained to prevent unrealistic influence on the graph structure. This approach ensured that the graph representation remained robust and reflective of real-world network dynamics.

The hyperparameters for each model were tuned using grid search to ensure fair comparisons. For a fair comparison with other models, we ensure that each baseline is configured using its respective default settings. For example, for GCN and GAT, we follow their original configurations in [37,4]. For E-GraphSAGE, we used the configuration in [18], where the number of hidden units is selected from the set {16, 32, 64, 128} with learning rate of 0.001. In the case of GCN-MHSA, we followed the settings in [10] where they set batch size to 64 and a dropout rate at 0.5 with epoch size as 50 and a learning rate as 0.01. This is consistent across models ensures that each model is tested under their best conditions. Both the inner and outer iterations for training, denoted as T_1 and T_2 , are set to 10 to allow for adequate convergence across models.

All experiments are conducted using Intel Xeon CPU with multiple cores. NVIDIA Tesla V100 GPUs are used to accelerate training,

which is critical for handling the large-scale graph structures and attention mechanisms in RAGN and baseline models. The system has 512 GB of RAM, allowing for efficient storage and computation of large intermediate data such as feature matrices, adjacency matrices, and attention scores. The experiments are conducted on a Linux-based system (Ubuntu 20.04). The models are implemented using PyTorch and DGL (Deep Graph Library) [38], which provide efficient handling of graph-based computations and allow for flexible implementation of graph neural networks. By running each experiment multiple times with different random seeds, we mitigate the potential for randomness in the results, providing a robust statistical basis for performance comparisons. This experimental setup allows us to fairly assess the ability of RAGN to detect and defend against adversarial attacks, comparing it against state-of-the-art baseline models under the same conditions.

4.2. Datasets and graph construction

For this study, we utilized the CICIDS-2018, CTU-13, and USTC-TFC2016 as benchmark datasets. Each model was trained, validated, and tested on the aforementioned datasets, with 80% of the data allocated for training, 10% for validation, and 10% for testing. The training set was used for learning the model parameters, while the validation set was employed for hyperparameter tuning. This split allows for optimal selection of model parameters without overfitting the training data. Detailed descriptions of these datasets are as follows:

1. CICIDS-2018 [39]: This dataset provides labeled traffic data that includes a range of different attacks, such as denial of service (DoS), botnet, and infiltration. From this dataset, we extracted relevant network features, including IP addresses, packet size, flow duration, and protocol type. We filtered out incomplete or erroneous flows, resulting in a clean set of labeled flows.
2. CTU-13 [40]: Known for its collection of botnet traffic, the CTU-13 dataset required careful selection and filtering of benign and malicious traffic flows. We utilized flow-based aggregation techniques to capture the behavior of bots and their interactions with other hosts. Non-malicious traffic was also preprocessed by grouping related flows into coherent sessions.

3. USTC-TFC2016 [41]: This dataset, which contains a variety of encrypted malicious traffic, provided challenges in distinguishing between malicious and benign flows based on encrypted data. We applied feature engineering techniques to extract meaningful attributes from encrypted traffic, such as packet timing and flow directionality, ensuring these flows were appropriately represented for graph construction.
4. UJS-IDS2022 dataset [10]: It is a private dataset comprising of real-life normal traffic and abnormal sourced from the Jiangsu Key Laboratory of Security Technology for Industrial Cyberspace in Jiangsu University.

The raw network traffic data was pre-processed to ensure compatibility with our graph-based model. For each dataset, the raw network traffic flows were first converted into flow-level features, representing individual connections between hosts in the network. A combination of packet-level and flow-level features (e.g., source IP, destination IP, packet counts, byte size, protocol types) were extracted to create a structured representation of each connection. Once the flow-level features were extracted from each dataset, we proceeded to construct a graph where each node represented a network entity (such as an IP address, a host, or a device). The edges between nodes were established based on observed traffic flows, capturing the interactions and communication patterns between the network entities. For each graph, each node in the graph corresponds to a unique entity within the network, such as an IP address or a network endpoint. The features of each node were derived from the corresponding flow-level features, including the number of connections, types of protocols used, packet sizes, and traffic volume. These features were critical in capturing the behavioral patterns of each network entity. Edges between nodes were defined by the presence of traffic between network entities. If a traffic flow existed between two nodes (e.g., an IP connection), an edge was created. The edge features were derived from the aggregated statistics of the flows, such as the total bytes transferred, the number of packets exchanged, and the duration of communication. Both directed and undirected graphs were constructed depending on the type of traffic (e.g., bidirectional traffic for TCP-based connections).

During data preprocessing, outlier handling was performed to ensure the quality and reliability of the input data. Outliers in numerical features, such as packet size and flow duration, were identified using the Interquartile Range (IQR) method. Data points falling outside the range $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ were excluded as outliers. For network-specific features, domain knowledge was applied to define thresholds. For example, unusually high connection rates (e.g., > 1000 connections/second from a single IP) were flagged and removed. These preprocessing steps were applied consistently across all datasets to maintain uniformity and ensure that the constructed graphs accurately reflected the underlying network traffic patterns.

In order to evaluate the robustness of RAGN against adversarial traffic, we introduced simulated adversarial perturbations to the graph structure by randomly adding or removing edges. This step allowed us to test how RAGN handles manipulated connections that could mislead the model. Each node was labeled as either benign or malicious, based on the ground-truth labels available in the dataset. In some cases, when the dataset provided flow-level labels (instead of node labels), we aggregated the flow labels to assign a label to the corresponding node. For example, if a host exhibited multiple malicious flows, it was labeled as malicious. Conversely, if a host only engaged in benign communication, it was labeled as benign. Each graph was carefully constructed to balance between sparsity and density, ensuring that the graph structure accurately reflected real-world network traffic without being overly connected or too sparse. This balance was important for ensuring that the attention mechanism in RAGN could focus on meaningful connections. By transforming these datasets into graph structures, we captured both the feature-level and relational information inherent in network traffic, which is crucial for the effective detection of malicious behaviors. The final graph representations were then used as input to our RAGN model, allowing it to learn from both the features of individual nodes and the communication patterns between them.

4.3. Evaluation metrics

We evaluate the performance of RAGN and the baseline models using four standard metrics in the field of intrusion detection. The metrics used in this study to evaluate the performance of the models are Detected Rate, Misclassification Rate, and Accuracy (ACC). These metrics are formulated as follows: The Detected Rate is the proportion of actual positive instances that were correctly identified by the model:

$$\text{Detected Rate} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}} \times 100 \quad (40)$$

The Misclassification Rate measures the proportion of incorrect classifications (both false positives and false negatives) in relation to the total number of instances:

Misclassification Rate

$$= \frac{\text{False Positives (FP)} + \text{False Negatives (FN)}}{\text{Total Instances}} \times 100 \quad (41)$$

The Accuracy (ACC) is the overall proportion of correct classifications, including both true positives and true negatives, out of the total instances:

$$\text{Accuracy (ACC)} = \frac{\text{True Positives (TP)} + \text{True Negatives (TN)}}{\text{Total Instances}} \times 100 \quad (42)$$

These metrics provide a comprehensive view of the model's performance in terms of its ability to correctly detect malicious traffic, avoid misclassifications, and maintain overall accuracy.

4.4. Baseline models

We compare the performance of RAGN against a set of state-of-the-art baseline models that are graph-based intrusion detection tasks:

1. GCN-MHSA [10]: Introduces a multi-head self-attention mechanism into GCN to assign larger weights to important features and introduces indirect and direct correlations between network traffic using link homogeneity.
2. GATrans [20]: A novel anomaly detection method that employs an enhanced graph attention network and Transformer model to account for temporal and spatial dependencies respectively.
3. FedGAT [42]: This model is based on an attention mechanism and the construction of a Federated Graph Attention Network to evaluate the interactivity between nodes in the graph, thereby improving the precision of internal network interactions. The method achieved comparable accuracy and robustness to traditional detection methods.
4. TCGNN [33]: This model converts each network packet into an undirected graph and utilizes a two-layer GCN that implements three distinct aggregation strategies. This approach effectively extracts the latent application representation from the graph-structured like a packet. Leveraging the robust capabilities of GNNs in graph representation learning, TCGNN achieves exceptionally high accuracy in identifying unknown network packets.
5. EC-GCN [43]: is an end-to-end classification model that learns representative spatial-temporal traffic features hidden in a traffic time series and then classifies them in a unified framework.
6. E-GraphSAGE [18]: It leverages Graph Neural Networks (GNNs) to detect and mitigate network-based cyber attacks by capturing both edge features and topological information of network traffic represented in a graph format.

To evaluate the effectiveness of SPAN, we also compare it against several benchmark attack methods that are commonly used in graph-based adversarial learning.

- DICE (Dynamic Iterative Classifier Exploitation) [44]: DICE is an attack method that iteratively manipulates node features and edges to degrade the classifier's performance. The approach works by exploiting the classifier's decision boundaries, aiming to force misclassification by perturbing nodes in a way that maximizes the classifier's error.
- DegAttack (DGA) [45]: DegAttack is a well-known attack method that focuses on degrading the graph by removing nodes with the highest degree. The method primarily targets highly connected nodes, which are considered structurally important, to reduce the overall connectivity of the graph.
- RWCS (Random Walk-based Community Structure) [36]: RWCS is an attack method that uses random walks to identify and disrupt community structures within a graph. By targeting nodes that are critical to maintaining community cohesion, RWCS aims to weaken the overall graph structure.

5. Experimental results

This section presents the experimental results of RAGN in terms of its effectiveness in detecting malicious network traffic and defending against adversarial attacks. The primary goal of the experiments is to assess RAGN's performance across different datasets, attack scenarios, and parameter configurations, demonstrating its advantages over existing methods. Additionally, we conduct ablation studies to highlight the impact of key components introduced in RAGN, such as the extra attention on edges and the iterative graph structure adjustment.

5.1. Parameter determination

In order to determine the optimal learning rate η_A for updating $\bar{A}$, and $\mathcal{P}_A$, we experimented the influence of different values. In the learning rate optimization experiment, we explored a range of learning rates for updating the adjacency matrix (η_A) and evaluated the model's performance based on both validation accuracy and convergence time. The results indicate that a learning rate of 0.001 yields a high validation accuracy of 89%, but requires a convergence time of 30 epochs. As the learning rate increases to 0.01, the validation accuracy remains high at approximately 88.7%, while the convergence time significantly decreases to 15 epochs, striking a balance between performance and training speed. However, further increasing the learning rate to 0.05 results in a notable decline in accuracy to around 86% and a convergence time of 13 epochs. At the highest learning rate of 0.1, the validation accuracy drops further to 82%, and convergence time reduces to 10 epochs. These findings suggest that overly aggressive updates can lead to suboptimal performance. The optimal learning rate for our model is 0.01, as it provides a favorable trade-off between validation accuracy and convergence speed, achieving high performance while maintaining efficient training time.

The experiment on graph structure adjustment strength (α) shows that moderate values of $\alpha = 0.05$ yield the best performance, achieving around 89% validation accuracy while maintaining a reasonable convergence time. Increasing α beyond this point reduces accuracy but accelerates convergence. Conversely, smaller α values improve accuracy but prolong convergence. The error bars indicate that model performance is most stable at $\alpha = 0.05$, with more variance at extreme values of α . Thus, a balanced adjustment strength optimizes both accuracy and convergence speed.

We conducted another experiment to evaluate the effect of the feature smoothing coefficient β on the model's performance. We tested values of β ranging from 0.01 to 1.0, specifically $\beta = 0.01, 0.05, 0.1, 0.5, 1.0$. The results show that the highest validation accuracy of 89.19% occurs at $\beta = 0.5$, indicating that moderate smoothing yields optimal results. Lower values of β , such as 0.01 and 0.05, lead to slightly reduced accuracy, while higher values, like $\beta = 1.0$, cause over-smoothing and

a decline in performance. Thus, the results highlight that a balanced feature smoothing approach is crucial for maximizing accuracy.

We also evaluate the impact of the adversarial loss coefficient λ on both validation accuracy and adversarial robustness. As shown in the chart, we varied λ across five different values: 0.01, 0.05, 0.1, 0.5, and 1. The results indicate that the model performs consistently well across different values of λ , with minimal fluctuations in validation accuracy and adversarial robustness. A slight drop is observed at $\lambda = 1$, which suggests that a very high value of λ may slightly hinder the model's overall performance. The adversarial robustness also follows a similar trend, reflecting the model's ability to maintain stability against adversarial attacks across a wide range of coefficients. These findings imply that the choice of λ can influence the balance between accuracy and robustness, but the model remains relatively stable within the tested range. (See Fig. 5.)

5.2. Detection of unknown malicious traffic

Unknown malicious network traffic encompasses network behaviors that are new attacks previously not seen by a detection systems. In this study, we employ the Stochastic Block Model (SBM) [35] and GraphGAN [46] to generate synthetic patterns that closely mimic real-world zero-day attack traffic. The SBM is utilized to produce one set of synthetic traffic by modeling the complex interactions within network traffic, thus providing a structured simulation of network behavior. On the other hand, GraphGAN employs a generator to create traffic flows from random noise and a discriminator that learns to differentiate between real and synthetic flows. After rigorous training, the GraphGAN generator yields another set of synthetic traffic that also aims to mirror zero-day attack scenarios. We conduct separate statistical validations for traffic generated by each model, as shown in Figs. 6 (SBM) and 7 (GraphGAN). These validations confirm that both the SBM and GraphGAN traffic closely mimic real-world patterns confirming quality samples for mimicking zero-day malicious traffic even though they do not perfectly replicate the complexity and unique features of real ones. These methodologies introduce meaningful variations as in zero-day attack behavior, ensuring that the model does not overfit to predefined attack signatures.

The models were then trained exclusively on the original dataset, excluding any synthetic traffic, to simulate real-world conditions where zero-day attacks represent entirely novel threats.

In the summarized results in Table 4, the proposed RAGN outperformed all baseline models across various datasets when challenged with zero-day traffic samples. Specifically, RAGN achieved an accuracy, precision, recall, and F1-score of 99.17%, 99.21%, 99.13%, and 99.16% respectively on the CICIDS-2018 dataset. These figures are significantly higher compared to the best-performing baseline model on the same dataset, TCGNN, which scored 89.19%, 88.13%, 87.58%, and 87.84% in accuracy, precision, recall, and F1-score, respectively.

On the GraphGAN version of the CICIDS-2018 dataset, RAGN maintained high scores with 99.12% accuracy, 99.18% precision, 99.09% recall, and 99.13% F1-score. In contrast, the TCGNN, which performed best among the baseline models under simulated conditions, showed a decrease to 85.27% accuracy, 83.82% precision, 83.21% recall, and 83.52% F1-score. This stark contrast in performance illustrates the effectiveness of RAGN's adaptive mechanisms, particularly its ability to maintain robustness against new and evolving threat patterns. The differential performance between RAGN and the best baseline models is approximately 10% to nearly 15% across all metrics in the simulated settings. On the UJS-2022 dataset, in scenarios involving unknown attack traffic, simulated using GraphGAN and SBM methods, RAGN exhibit robust performance, maintaining high accuracy (98.12% and 98.11%) and precision (98.18% and 98.16%), thereby illustrating its resilience and adaptive capability against novel adversarial strategies. Furthermore, RAGN also records the highest recall rates among

Table 4
Zero-day malicious traffic detection performance.

Dataset	Metric (%)	Original data							Unknown attack traffic (GraphGAN)							Unknown attack traffic (SBM)						
		GCN	GATr	FedGAT	EC-GCN	TCGNN	E-GrSAGE	RAGN	GCN	GATr	FedGAT	EC-GCN	TCGNN	E-GrSAGE	RAGN	GCN	GATr	FedGAT	EC-GCN	TCGNN	E-GrSAGE	RAGN
CICIDS-2018	Accuracy	83.57	84.63	85.28	86.34	89.19	88.12	99.17	79.18	80.37	81.42	82.53	85.27	84.10	99.12	78.24	79.38	80.67	81.59	84.35	83.18	99.11
	Precision	82.22	83.56	84.37	85.41	88.13	87.06	99.21	77.05	78.54	79.83	80.78	83.82	82.67	99.18	76.13	77.68	78.94	79.89	82.73	81.59	99.16
	Recall	81.53	82.98	83.77	84.82	87.58	86.51	99.13	76.23	77.67	79.16	80.02	83.21	82.05	99.09	75.84	77.29	78.73	79.63	82.34	81.20	99.07
	F1-Score	81.89	83.24	84.06	85.18	87.84	86.78	99.16	76.62	78.02	79.49	80.44	83.52	82.36	99.13	75.99	77.49	78.84	79.76	82.56	81.42	99.11
CTU-13	FPR	10.57	10.24	9.83	9.54	7.42	7.90	0.32	12.73	12.51	11.82	11.52	9.79	10.22	0.35	12.68	12.43	11.88	11.48	9.77	10.30	0.34
	Accuracy	82.69	83.47	85.12	86.25	89.28	88.21	98.95	81.08	82.12	83.56	84.78	88.05	87.02	98.90	80.21	81.32	82.64	83.83	86.89	85.86	98.88
	Precision	81.53	82.85	84.52	85.83	88.92	87.85	98.89	80.58	81.85	83.04	84.27	87.65	86.58	98.85	79.67	80.95	82.17	83.34	86.45	85.42	98.83
	Recall	80.95	82.35	84.05	85.33	88.45	87.38	98.86	79.08	80.35	81.58	82.75	86.15	85.11	98.82	78.19	79.47	80.69	81.90	85.20	84.17	98.80
USTC-TFC2016	F1-Score	81.25	82.60	84.30	85.60	88.70	87.61	98.87	79.83	81.10	82.31	83.52	86.90	85.84	98.83	78.90	80.20	81.43	82.62	85.82	84.79	98.81
	FPR	10.35	10.05	9.55	9.25	7.65	8.15	0.29	11.55	11.25	10.75	10.45	8.95	9.50	0.31	11.50	11.25	10.70	10.40	8.90	9.45	0.30
	Accuracy	88.13	89.04	90.59	91.39	95.17	94.10	99.53	87.58	88.47	89.97	91.15	94.02	93.00	99.48	86.28	87.18	88.71	89.91	92.29	91.25	99.45
	Precision	87.55	88.45	89.95	90.75	94.82	93.75	99.48	86.08	87.07	88.47	89.67	92.83	91.81	99.43	85.08	86.17	87.57	88.77	91.03	90.00	99.40
UJS-IDS2022	Recall	87.05	88.05	89.55	90.35	94.45	93.38	99.45	85.58	86.57	88.07	89.27	92.57	91.55	99.40	84.78	85.87	87.32	88.52	90.78	89.73	99.38
	F1-Score	87.30	88.25	89.75	90.55	94.65	93.56	99.47	85.83	86.82	88.32	89.52	92.70	91.68	99.42	84.93	86.01	87.45	88.65	90.92	89.86	99.39
	FPR	8.43	8.13	7.73	7.33	3.93	4.15	0.28	9.03	8.83	8.33	8.03	5.63	5.85	0.30	8.98	8.78	8.28	7.98	5.58	5.80	0.29
	Accuracy	90.57	91.63	92.28	93.34	96.19	95.12	99.17	86.18	87.37	88.42	89.53	92.27	91.10	98.12	85.24	86.38	87.67	88.59	91.35	90.18	98.11
UJS-IDS2022	Precision	89.22	90.56	91.37	92.41	95.13	94.06	98.21	84.05	85.54	86.83	87.78	90.82	89.67	98.18	83.13	84.68	85.94	86.89	89.73	88.59	98.16
	Recall	88.53	89.98	90.77	91.82	94.58	93.51	98.13	83.23	84.67	86.16	87.02	90.21	89.05	98.00	82.84	84.29	85.73	86.63	89.34	88.20	97.87
	F1-Score	88.89	90.24	91.06	92.18	94.84	93.78	98.16	83.62	85.02	86.49	87.44	90.52	89.36	97.13	82.99	84.49	85.84	86.76	89.56	88.42	97.11
	FPR	9.57	9.24	8.83	8.54	6.42	6.90	0.22	11.73	11.51	10.82	10.52	8.79	9.22	0.25	11.68	11.43	10.88	10.48	8.77	9.30	0.24

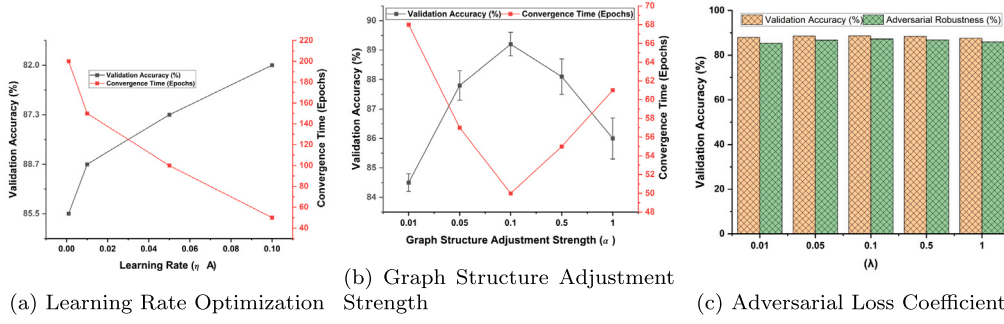


Fig. 5. Parameter determination experiments.

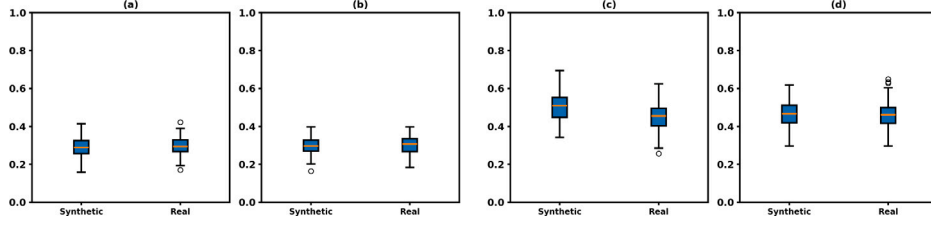


Fig. 6. Box plots were utilized to analyze features such as (a) Bwd Packet Length Max, (b) Fwd Packets Length Total, (c) Bwd Packet Length Max, and (d) Init Fwd Win Bytes in the CICIDS-2018 datasets. The presented box plots illustrate the disparities between real and synthetic data, leading to the termination of the GANs training process.

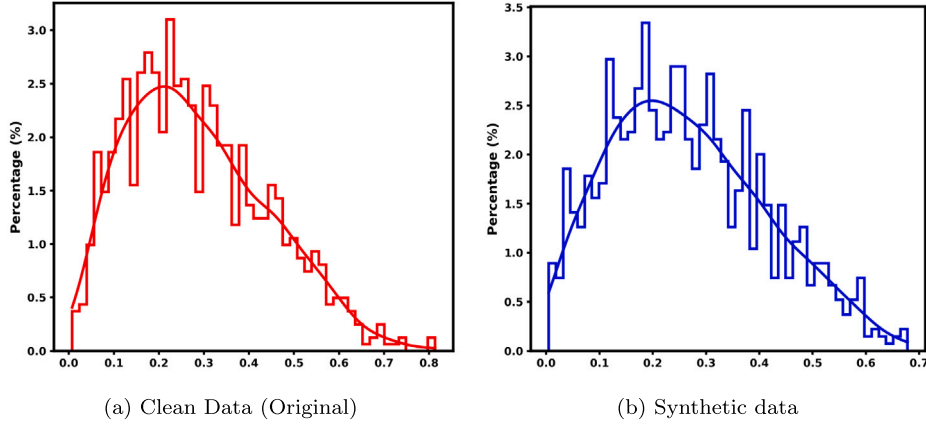


Fig. 7. Jaccard similarity distributions between clean and GraphGAN's synthetic datasets.

all models, achieving 98.13% on the original dataset and maintaining commendable performance under attack simulations (98.00% for GraphGAN and 97.87% for SBM). The superior F1-scores (98.16% in original, 97.13% in GraphGAN, and 97.11% in SBM scenarios) suggest a balanced high performance between precision and recall, which is challenging to achieve in adversarial settings. Additionally, RAGN demonstrates an exceptionally low False Positive Rate (FPR) of 0.22% in the original data scenario, with only a slight increase to 0.25% and 0.24% under GraphGAN and SBM attacks, respectively.

Such a robust performance by RAGN not only highlights the model's advanced architectural merits but also its practical applicability in real-world network environments where adaptability and accuracy are paramount. The ability of RAGN to sustain low false positive rates across all datasets further asserts its operational efficacy, a crucial factor for systems deployed in sensitive and critical network infrastructures.

5.3. Detection performance under different adversarial attacks

In this experiment, we evaluated the resilience of the detection models against four distinct adversarial attack methods. These included

SPAN, a dynamic graph attack we introduced in this study, along with DICE [44], degattack (DGA) [45], and RWCS [36]. The adversarial perturbation levels varied from 2% to 25%, aimed at disrupting the graph structure. The comparative performance, as detailed in Table 5, illustrates the superior resilience of RAGN across all attack vectors on the CICIDS-2018 Dataset. Under the DICE attack, all baseline models exhibited higher misclassification rates as the perturbation ratio increased. However, RAGN consistently outperformed the other models, maintaining significantly lower misclassification rates at all perturbation levels. For instance, at a 25% perturbation rate, RAGN achieved a misclassification rate of 7.06%, substantially lower than the next best model, FedGAT, which recorded 17.02%. This trend was similarly observed under the DGA attack, where RAGN showcased exceptional robustness, maintaining a misclassification rate of just 1.16% at 5% perturbation and 4.88% at 25%, in stark contrast to other models that approached or exceeded 15%. The RWCS attack further validated RAGN's resilience, indicating its capability to effectively counter complex adversarial manipulations. Even at lower perturbation levels of 2%, RAGN recorded the lowest misclassification rate of 1.34%, and this strong performance was sustained as perturbations increased, with rates of 3.80% at 20% and 4.40% at 25%. The SPAN attack, targeting

Table 5

Attack performance (misclassification rate) on CICIDS-2018 dataset.

Attack method	PTR (%)	GCN	GATr	FedGAT	EC-GCN	TCGNN	E-GrSAGE	RAGN
DICE	2	2.05	1.98	1.89	1.94	1.84	1.81	1.22
	5	6.13	5.94	5.81	5.58	5.30	5.14	4.56
	10	10.17	9.82	9.55	9.16	8.86	8.58	5.28
	15	14.10	13.56	13.03	12.42	12.05	11.43	6.10
	20	17.83	17.15	16.52	15.83	15.31	14.48	6.12
	25	21.53	20.63	19.78	18.81	18.07	17.02	7.06
DGA	2	1.97	1.93	1.85	1.82	1.74	1.68	1.00
	5	5.93	5.67	5.49	5.30	5.06	4.85	1.16
	10	9.50	9.19	8.86	8.58	8.27	7.95	1.88
	15	13.22	12.63	12.19	11.63	11.16	10.64	3.20
	20	16.99	16.27	15.59	14.92	14.20	13.32	3.24
	25	20.64	19.63	18.72	17.65	16.65	15.59	4.88
RWCS	2	2.12	2.06	1.97	1.90	1.81	1.73	1.34
	5	6.33	6.11	5.91	5.66	5.44	5.22	2.50
	10	10.46	10.11	9.76	9.39	9.04	8.59	2.90
	15	14.59	13.97	13.34	12.78	12.09	11.47	3.72
	20	18.56	17.84	17.01	16.15	15.18	14.12	3.80
	25	22.35	21.32	20.23	19.18	17.88	16.69	4.40
SPAN	2	2.01	1.95	1.87	1.80	1.72	1.63	1.14
	5	5.98	5.71	5.55	5.40	5.12	4.88	2.12
	10	9.79	9.44	9.07	8.76	8.39	7.98	2.50
	15	13.83	13.16	12.61	12.08	11.42	10.76	3.14
	20	17.95	16.88	16.17	15.31	14.38	13.30	3.40
	25	21.83	20.77	19.62	18.49	17.11	15.94	4.50

Table 6

Attack performance (misclassification rate) on CTU-13 dataset.

Attack method	PTR (%)	GCN	GATr	FedGAT	EC-GCN	TCGNN	E-GrSAGE	RAGN
DICE	2	1.20	1.16	1.10	1.06	1.00	0.98	0.92
	5	3.30	3.18	3.04	2.92	2.82	2.74	2.46
	10	5.36	5.14	4.98	4.76	4.58	4.38	4.08
	15	7.38	7.06	6.76	6.40	6.16	5.84	5.42
	20	9.24	8.88	8.50	8.04	7.62	7.18	6.56
	25	11.10	10.60	10.08	9.50	8.84	8.16	7.50
DGA	2	1.16	1.12	1.06	1.00	0.94	0.90	0.84
	5	3.18	3.06	2.96	2.84	2.70	2.58	2.36
	10	5.14	4.96	4.78	4.54	4.32	4.14	3.84
	15	7.10	6.76	6.48	6.10	5.74	5.36	4.94
	20	8.94	8.52	8.08	7.58	7.04	6.48	5.96
	25	10.72	10.16	9.58	8.84	8.10	7.40	6.70
RWCS	2	1.24	1.18	1.12	1.06	1.00	0.96	0.88
	5	3.36	3.22	3.12	2.98	2.82	2.70	2.44
	10	5.42	5.20	4.96	4.72	4.50	4.24	3.94
	15	7.46	7.12	6.80	6.42	6.04	5.62	5.16
	20	9.36	8.98	8.54	8.04	7.50	6.90	6.34
	25	11.24	10.66	10.00	9.26	8.48	7.80	7.12
SPAN	0	1.18	1.14	1.08	1.02	0.96	0.92	0.86
	5	3.24	3.10	3.00	2.86	2.70	2.56	2.32
	10	5.28	5.06	4.84	4.58	4.36	4.10	3.76
	15	7.36	7.02	6.70	6.32	5.88	5.46	5.04
	20	9.30	8.88	8.44	7.88	7.30	6.74	6.16
	25	11.16	10.58	9.92	9.24	8.40	7.72	6.98

dynamic graph structures, posed a considerable challenge; yet, RAGN demonstrated substantial resilience. Starting from a misclassification rate of 1.14% at 2% perturbation, RAGN effectively managed the threat, capping the misclassification rate at 4.50% at the highest perturbation level of 25%. This performance starkly contrasts with other models, which exhibited higher vulnerabilities under similar attack conditions.

For the CTU-13 dataset, we further examined the resilience of the detection models against the same four adversarial attacks as previously discussed: DICE [44], DGA [45], RWCS [36], and SPAN. The results depicted in Table 6 clearly illustrate the robust performance of our RAGN model in comparison to other models across varied levels of perturbation from 2% to 25%. With the DICE attack, RAGN showed

Table 7

Attack performance (misclassification rate) on USTC-TFC2016 dataset.

Attack method	PTR (%)	GCN	GATr	FedGAT	EC-GCN	TCGNN	E-GrSAGE	RAGN
DICE	2	1.20	1.15	1.10	1.05	1.00	0.97	0.462
	5	3.75	3.37	3.25	3.17	3.05	2.95	1.3625
	10	5.75	5.57	5.42	5.25	5.25	4.85	2.25
	15	8.05	7.72	7.35	7.02	6.80	6.45	2.95
	20	10.35	9.87	9.35	8.80	8.275	7.70	3.55
	25	12.55	11.95	11.25	10.35	9.55	8.77	3.95
DGA	2	1.125	1.075	1.25	1.00	0.95	0.925	0.43
	5	3.35	3.22	3.10	3.00	2.85	2.77	1.26
	10	5.625	5.40	5.20	5.00	4.75	4.57	2.13
	15	7.875	7.50	7.10	6.75	6.37	5.95	2.75
	20	10.10	9.62	9.10	8.55	7.95	7.30	3.37
	25	12.30	11.62	10.85	10.02	9.125	8.35	3.76
RWCS	2	1.25	1.20	1.15	1.07	1.25	0.95	0.45
	5	3.55	3.45	3.32	3.22	3.10	2.95	1.35
	10	5.95	5.72	5.50	5.27	5.25	4.75	2.21
	15	8.75	7.85	7.52	7.10	6.65	6.15	2.85
	20	10.50	9.95	9.45	8.85	8.20	7.55	3.43
	25	12.75	12.10	11.32	10.47	9.52	8.65	3.92
SPAN	2	1.17	1.125	1.05	1.00	0.95	0.95	0.42
	5	3.50	3.35	3.25	3.10	2.97	2.85	1.31
	10	5.75	5.65	5.45	5.25	4.97	4.67	2.18
	15	8.22	7.80	7.35	6.95	6.50	6.00	2.76
	20	10.45	9.90	9.35	8.75	8.25	7.37	3.37
	25	12.70	12.00	11.25	10.30	9.75	8.52	3.86

outstanding robustness by consistently maintaining lower misclassification rates compared to its counterparts. At a minimal perturbation rate of 2%, RAGN recorded a misclassification rate of 0.92%, which remains lower than the other models. As the perturbation rate increased to 25%, RAGN still outperformed with a rate of 6.56%, while the closest competitor model, TC-GNN, registered at 7.18%.

Under the DGA attack, the misclassification rates for RAGN started at an impressively low 0.84% at a 2% perturbation level, gradually increasing to 5.96% at 25% perturbation. This trend of lower misclassification rates signifies RAGN's effective handling of complex attack scenarios. For the RWCS attack, RAGN again demonstrated its effectiveness by starting with a misclassification rate of 0.88% at 2% perturbation and ending at 6.34% at 25%. This performance is notably better than the other models, which showed higher susceptibility to this type of attack. The SPAN attack results further demonstrated the resilience of RAGN in the dynamic attack environment. At 2% perturbation, RAGN achieved a misclassification rate of 0.86%, escalating only to 6.98% at the highest perturbation level of 25%. This outperforms other models by a significant margin, particularly under higher levels of perturbation.

The evaluation on the USTC-TFC2016 dataset, as summarized in Table 7, presents a comprehensive assessment of RAGN's capability to withstand adversarial conditions when compared to other prominent models. The assessment was conducted using four different attack methods: DICE [44], DGA [45], RWCS [36], and SPAN, with perturbation ratios ranging from 2% to 25%. In the context of the DICE attack, RAGN demonstrated remarkable robustness by maintaining significantly lower misclassification rates across all levels of perturbation. Notably, at 2% perturbation, the RAGN model achieved a misclassification rate of only 0.462%, and even at the extreme of 25% perturbation, it upheld a rate of 3.95%, substantially lower than the rates observed in other models, where misclassification escalated beyond 10%. Under the DGA attack, RAGN consistently showcased superior performance, initiating with a misclassification rate of 0.43% at 2% perturbation and concluding at 3.76% at 25% perturbation. This indicates a resilient performance trend that starkly contrasts with the next best performing model, E-GSAGE, which concluded at 8.35% at the same perturbation level. The RWCS attack results further solidified RAGN's leading position, starting at 0.45% misclassification at 2% perturbation

Table 8

Attack performance (misclassification rate) on UJS-IDS2022 dataset.

Attack method	PTR (%)	GCN	GATr	FedGAT	EC-GCN	TCGNN	E-GrSAGE	RAGN
DICE	2	0.95	0.90	0.87	0.85	0.80	0.78	0.35
	5	2.80	2.60	2.50	2.40	2.30	2.20	1.05
	10	4.50	4.35	4.20	4.05	3.95	3.85	1.78
	15	6.75	6.50	6.25	6.00	5.75	5.50	2.50
	20	8.90	8.60	8.30	8.00	7.70	7.40	3.30
	25	11.00	10.75	10.50	10.25	10.00	9.75	4.00
DGA	2	0.92	0.87	0.85	0.83	0.81	0.79	0.34
	5	2.65	2.55	2.45	2.35	2.25	2.15	1.00
	10	4.40	4.25	4.10	3.95	3.80	3.65	1.70
	15	6.60	6.35	6.10	5.85	5.60	5.35	2.40
	20	8.75	8.45	8.15	7.85	7.55	7.25	3.20
	25	10.85	10.60	10.35	10.10	9.85	9.60	3.90
RWCS	2	1.00	0.95	0.92	0.90	0.88	0.86	0.40
	5	3.00	2.90	2.80	2.70	2.60	2.50	1.15
	10	4.95	4.80	4.65	4.50	4.35	4.20	1.90
	15	7.15	6.90	6.65	6.40	6.15	5.90	2.65
	20	9.20	8.95	8.70	8.45	8.20	7.95	3.50
	25	11.25	11.00	10.75	10.50	10.25	10.00	4.25
SPAN	2	0.97	0.92	0.90	0.88	0.86	0.84	0.38
	5	2.75	2.65	2.55	2.45	2.35	2.25	1.10
	10	4.70	4.55	4.40	4.25	4.10	3.95	1.85
	15	6.85	6.60	6.35	6.10	5.85	5.60	2.60
	20	9.00	8.75	8.50	8.25	8.00	7.75	3.45
	25	11.15	10.90	10.65	10.40	10.15	9.90	4.20

and ending at 3.92% at 25%. The consistent low misclassification rates emphasize RAGN's effectiveness in neutralizing more complex adversarial threats. Similarly, for the SPAN attack, RAGN began with a misclassification rate of 0.42% at 2% perturbation and ended at 3.86% at 25%, maintaining the lowest misclassification rates across all models. This performance is particularly noteworthy given the dynamic and sophisticated nature of the SPAN attack, designed to challenge the adaptability of dynamic graph structures.

For the UJS-IDS2022 dataset, the results illustrated in Table 8 show that at a 2% perturbation rate, RAGN achieved a misclassification rate of 0.35%, which is significantly lower than the next best model (E-GrSAGE at 0.78%). This trend continues across all perturbation rates, with RAGN maintaining a lower misclassification rate, culminating at 4.00% at a 25% perturbation rate, compared to 9.75% for the least resilient model under the same conditions. These findings show the architectural strengths of RAGN, particularly its ability to handle adversarial modifications more effectively than other tested models.

The lower susceptibility of RAGN to adversarial attacks can be attributed to its sophisticated integration of node features and dynamic adaptations to graph structural changes over time. This capability is crucial for applications in environments where data integrity and reliability are paramount, such as cybersecurity and network traffic monitoring. The results suggest that further improvements and optimizations in dynamic graph neural network architectures could enhance their defense mechanisms against increasingly complex adversarial strategies, thereby bolstering their application in security-critical systems.

5.4. Evaluation in time evolving dynamic environments

This experiment was designed to test the detection models' adaptability to dynamic graph attacks based on the proposed SPAN attack method. We used two large-scale proprietary datasets (i.e., the USTC-TFC2016 and the UJS-IDS2022 datasets) for this experiment. The experiment began with an unperturbed graph, $G_0 = (V_0, A_0, F_0)$, representing the baseline network state. At each subsequent time step, the graph evolved into $G_t = (V_t, A_t, F_t)$ through structural perturbations and temporal updates. New nodes were injected into the graph at each time step, simulating the arrival of devices or endpoints in the network.

These nodes were initialized with features sampled from the same distribution as existing nodes in V_t , ensuring that the new additions were statistically and semantically consistent with the original dataset. Simultaneously, existing nodes were removed to mimic device failures or targeted disruptions.

The removal process followed two strategies: a random selection, where nodes were chosen uniformly across the graph, and a targeted selection, where high-degree or high-centrality nodes were prioritized for removal to amplify the impact on the graph structure. In addition to node dynamics, edges were also incrementally added and removed. The newly added edges connected nodes based on the formula of structural fragility score (see formula (34)), where $\deg(u)$ and $\deg(v)$ represent the degrees of the connecting nodes. This scoring mechanism ensured that newly added edges targeted structurally fragile or underutilized regions of the graph, thereby introducing meaningful disruptions to its structure. The edge removal also followed both a random and a targeted removal approach, which focused on removing connections between critical nodes or edges with high betweenness centrality to simulate link disruptions. Temporal consistency was maintained throughout the experiment to ensure realistic transitions between graph states. Newly added nodes and edges were integrated into the graph in a manner consistent with the temporal properties of the network, while removed nodes and edges were excluded dynamically without causing abrupt disruptions. 10% and 25% perturbations were applied over time steps of the graph's total nodes or edges, reflecting gradual network evolution.

The detection models were then evaluated at each time step on the evolving graph G_t without retraining, allowing for an assessment of its ability to generalize to unseen dynamic changes. The averaged results over five independent runs are summarized in Fig. 8. On node perturbation, Figs. 8(c), and 8(d) show the results for the random node removal approach while Figs. 8(e) and 8(f) show results for targeted nodes for UJS-IDS2022 and USTC-TFC2016 datasets respectively. The results present the performance of the detectors across the different timestamps and at 10% and 25% perturbation rates. The results are discussed in the following subsections.

5.4.1. Time-evolving graph node perturbation

For random Node Removal, in the UJS-IDS2022 dataset, at a 10% perturbation rate, accuracy rates for RAGN are notably higher, maintaining around 95% to 89.6%, and for the USTC-TFC2016 dataset, RAGN shows similar resilience. At a 25% perturbation rate, RAGN's accuracy remains significantly higher than that of baseline models, staying around 94.6% to 88.6%, as depicted in Figs. 8(c), and 8(d). Under targeted node removal, RAGN demonstrates superior performance compared to baseline models. In the UJS-IDS2022 dataset, RAGN maintains an accuracy rate of approximately 97.4% to 90.4% at a 10% perturbation, which only slightly decreases to about 92.9% to 85.9% at 25% perturbation, as shown in Fig. 8(e). The USTC-TFC2016 dataset reveals a similar pattern, with RAGN achieving higher accuracy than the baselines, maintaining 97.4% to 90.4% at 10% perturbation and around 87.5% to 85% at 25% perturbation, as highlighted in Fig. 8(f).

For node Injection, the scenario of node injection showcases RAGN's robustness where, even with new node introductions, it outperforms baselines significantly. For the UJS-IDS2022 dataset, RAGN maintains accuracy rates well above 87% at 10% perturbation, and around 94% to 86% at 25% perturbation, as illustrated in Fig. 8(a). The USTC-TFC2016 dataset shows similar trends, where RAGN's accuracy decreases less dramatically than that of the baseline models, particularly at higher perturbation levels, as seen in Fig. 8(b). Across all tests, including random and targeted node perturbations as well as node injections, RAGN consistently outperforms baseline models like GCN, GAT, EC-GCN, and TC-GNN. RAGN exhibits a more gradual decrease in accuracy with increasing perturbation rates, demonstrating its robustness and adaptability. These findings suggest the superiority of RAGN in maintaining high performance under adversarial conditions and indicate its potential for deployment in critical network environments where resilience to adversarial attacks is paramount.

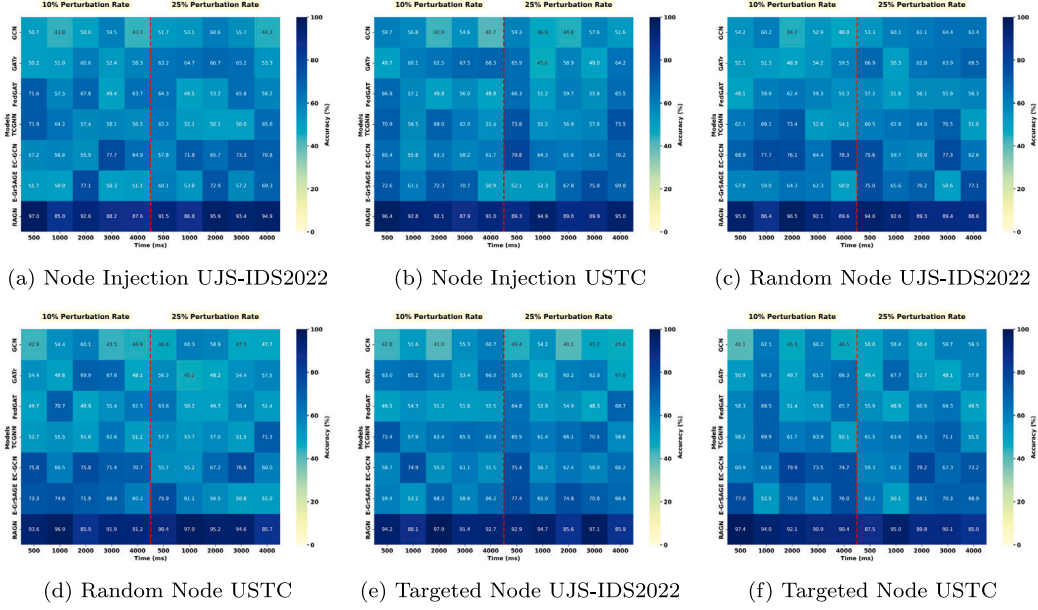


Fig. 8. Model performance in different timestamps under different node perturbation rates.

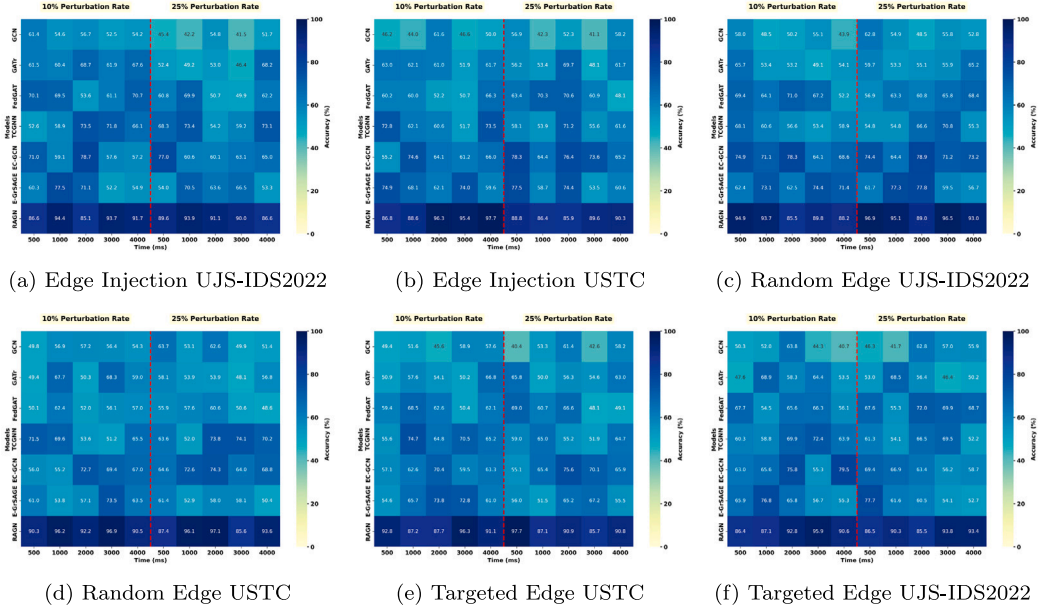


Fig. 9. Model performance at different timestamps under different edge perturbation rates.

5.4.2. Time-evolving graph edge perturbation

This experiment evaluated the resilience of the detection models against edge perturbations across different models revealing distinct insights into the resilience of RAGN compared to other models when challenged with the SPAN attack. In the UJS-IDS2022 dataset, RAGN demonstrates exceptional stability, maintaining high accuracy rates consistently above 90% across different timestamps at a 10% perturbation rate, and showing minimal performance degradation even as perturbation increases to 25%. This superior performance is reflected in both the Edge Injection and Random Edge scenarios, as depicted in Figs. 9(a) and 9(c), where RAGN significantly outperforms models like GCN, GAT, and FedGAT.

In the USTC-TFC2016 dataset, RAGN similarly exhibits robust performance, maintaining high accuracy across varying timestamps. At a 10% perturbation rate, RAGN's accuracy starts high and shows little

reduction, contrasting sharply with the performance dips seen in other models under the same conditions. This trend continues even under 25% perturbation, where RAGN's accuracy impressively remains above 85%, as illustrated in Figs. 9(b) and 9(d).

Targeted edge perturbations further demonstrates RAGN's capabilities in managing strategic disruptions effectively. In the UJS-IDS2022 dataset, despite the aggressive nature of targeted attacks, RAGN's accuracy only slightly decreases, maintaining a high level of performance throughout the experiment. This resilience is similarly observed in the USTC-TFC2016 dataset, where RAGN consistently performs better than the baseline models, particularly under higher perturbation rates, as shown in Figs. 9(f) and 9(e). RAGN's consistent high performance in both random and targeted edge perturbations highlights its potential for deployment in critical network environments where robustness against sophisticated adversarial attacks is paramount.

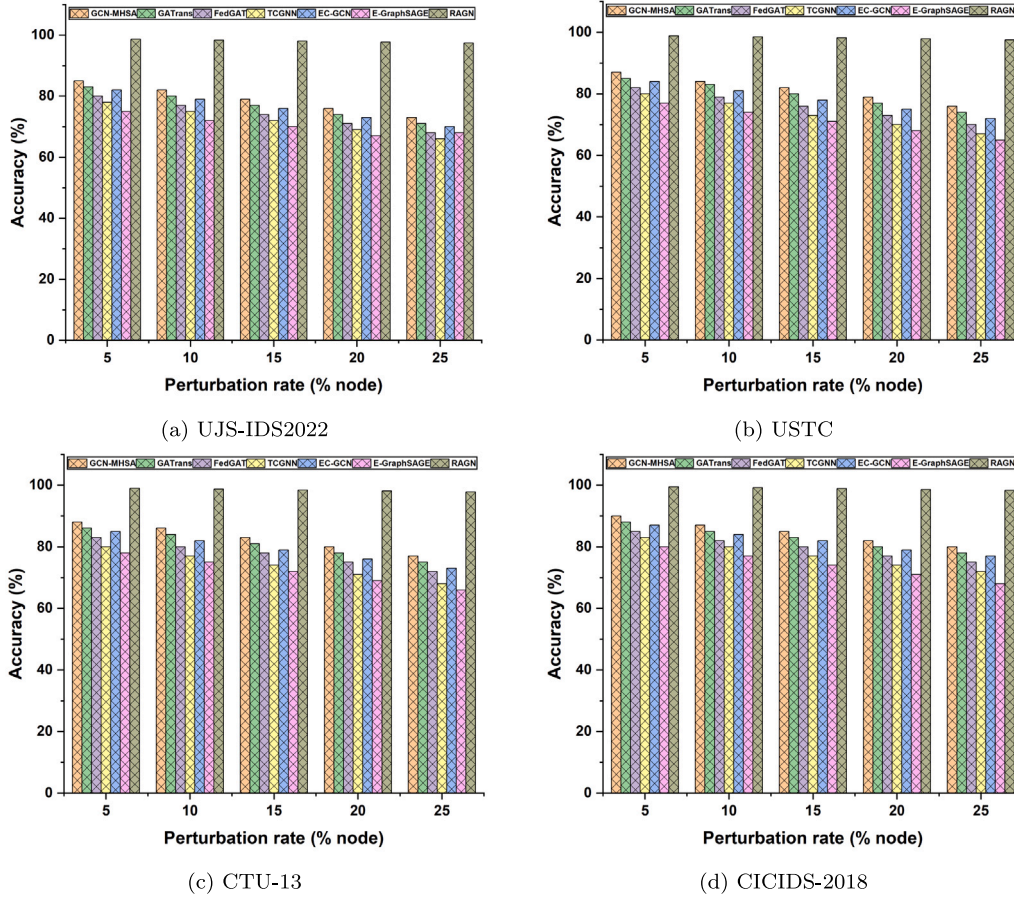


Fig. 10. Detection performances under malicious-based node injection and feature generation attack.

5.5. Malicious node injection attack

In this subsection, we conducted node classification experiments to assess the effectiveness of the proposed SPAN attack method on dynamic GNNs. We use RAGN and the baseline models as targets to evaluate their resilience against malicious node injection at different perturbation rates. The SPAN method injects malicious nodes at strategically vulnerable times and locations within the network graph to disrupt traffic analysis. Once the nodes are injected, SPAN generates adversarial features for these nodes using an optimization function designed to maximize the disruption to the target model's normal operations. By targeting weak points in the network's structure, SPAN degrades the performance of systems designed to detect malicious traffic.

The results in Fig. 10(a) show that for the UJS-IDS2022 dataset, the accuracy of the baseline models generally decreases from above 80% at a 5% perturbation rate to below 70% at a 25% perturbation rate. Similar trends are observed across the other datasets, though the initial and final accuracy levels vary, suggesting that the inherent characteristics of each dataset – such as the typical patterns of normal and malicious traffic – affect how susceptible they are to the SPAN attack.

RAGN, the proposed model in this study, shows relative resilience compared to the other models. For example, in the UJS-IDS2022 dataset, while other models' accuracy dips significantly at a 25% perturbation rate – falling to as low as around 63% – RAGN maintains over 90%. Similarly, on the USTC-TFC2016 dataset, RAGN's accuracy at a 5% perturbation rate begins near 98% and decreases to near 97% at 25%, which is notably higher than some competing models like GCN-MHSA and FedGAT which drop to around 65%.

This comparative robustness of RAGN may be attributed to its architectural features or the specific way it processes and integrates node features, which may provide a buffer against the disruptive effects of adversarial strategies deployed by dynamic graph attack methods. Furthermore, models like EC-GCN and EGraphSAGE demonstrate better performance under higher perturbation conditions across different datasets, suggesting that certain models may have inherent or designed-in resistances to specific types of adversarial manipulations used in SPAN. This resilience could stem from how these models manage node connectivity and feature integration, which might be less susceptible to the influence of adversarially injected nodes and their features.

5.6. Targeted label attack

A targeted label attack is defined as a type of adversarial manipulation where the adversary aims to mislead a graph neural network (GNN) into incorrectly classifying a specific node into a predefined, erroneous category. This type of attack is more strategic and malicious compared to untargeted attacks because it degrades the overall performance of the model and also forces specific, deliberate errors in node classification. In this experiment, we implement the effective targeted label attack (ETLA) [47] to manipulate graph structures and test the malicious traffic detection performance of RAGN and the baseline models under this attack condition. The ETLA method launches attacks by strategically selecting and modifying edges in a graph's structure to mislead a graph neural network into classifying a target node into a specific category. It uses a combination of stratified node selection, adaptive edge choice strategies, and a temperature-scaled attack loss function to allocate the attack budget and achieve the desired misclassification effectively.

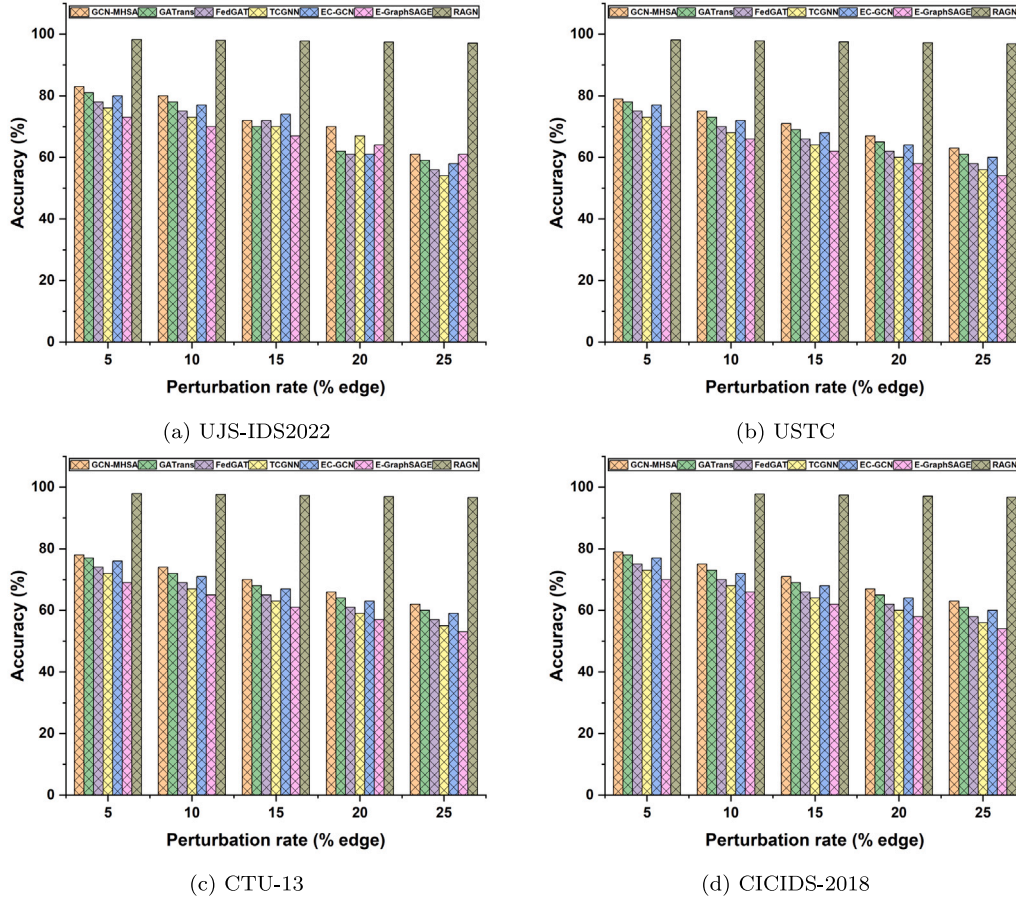


Fig. 11. Detection performance under Targeted label attack across Datasets.

From the results in Fig. 11, RAGN maintains an accuracy significantly higher than the baseline models even as the perturbation rate increases to 25%. For instance, in the UJS-IDS2022 dataset, RAGN retains an accuracy close to 95% at 25% perturbation, while other models like TCGNN and FedGAT fall below 60%. Similar trends are observable in the USTC-TFC2016 dataset, where RAGN outperforms counterparts with a consistent margin, holding an accuracy above 90%, contrasting sharply with others that drop to near 58%. In more sensitive datasets like CTU-13 and CICIDS-2018, where adversarial attacks generally cause a more drastic decline in performance, RAGN demonstrates its robustness effectively. For example, in the CTU-13 dataset, while the accuracy for models like GraphSAGE and RAGN dips as perturbations increase, RAGN's accuracy remains approximately 10% higher than GraphSAGE at a 25% perturbation rate, hovering around 97% compared to GraphSAGE's 61%. This pattern repeats in CICIDS-2018, indicating a strong adaptive capability inherent to RAGN that mitigates the impact of graph structural changes more efficiently than other models. The substantial decline in accuracy across baseline models with increasing perturbation rates highlights the effectiveness of ETLA's stratified node selection and adaptive edge choice strategies in disrupting the graph's structure. These strategies complicate the task of accurately classifying nodes for many networks. In contrast, RAGN's architecture demonstrates its effectiveness, adaptability, and resilience in maintaining higher accuracy under similar conditions.

5.7. Scalability analysis

The scalability evaluation of our approach was conducted using GraphGAN-generated network structures with sizes ranging from 100 to 100,000 nodes. The performance metrics assessed include execution

time, CPU utilization, memory consumption, throughput, and accuracy. The results, as presented in Table 9, demonstrate that the proposed method maintains computational efficiency while ensuring high accuracy across different network scales. The execution time exhibits a logarithmic growth pattern as network size increases, confirming that the method scales efficiently. For smaller networks, such as 100 nodes, the execution time is 22.60 s, indicating a lightweight and computationally feasible approach. As the network size increases, execution time rises gradually, reaching 42.62 s for 5000 nodes and 45.83 s for 10,000 nodes. Even for the largest tested network of 100,000 nodes, execution time remains controlled, ensuring that the method does not impose excessive processing costs. This trend suggests that the computational complexity of the method remains manageable, making it suitable for large-scale applications.

CPU utilization also follows a gradual increase as network size expands. For 100 nodes, CPU usage is 45.02%, increasing to 80.73% for 5000 nodes and reaching 92.18% for 10,000 nodes. The observed increase aligns with expectations, as larger networks require more processing power. However, the controlled growth ensures that the approach does not become prohibitively resource-intensive, making it feasible for real-world deployment. Similarly, memory utilization follows a predictable increase, starting at 89.54 MB for 100 nodes, 172.07 MB for 5000 nodes, and reaching 189.01 MB for 10,000 nodes. This controlled growth indicates that while the method demands additional memory as network size increases, it does not introduce severe memory bottlenecks. Throughput, measured in operations per second, shows an inverse relationship with execution time. For 100 nodes, throughput is 44.24 ops/sec, and as network size increases, it declines slightly to 23.46 ops/sec for 5000 nodes and 21.82 ops/sec for 10,000 nodes. Despite this decline, the results remain well within acceptable

Table 9
Scalability performance across different network sizes.

Network size	Execution time (s)	CPU utilization (%)	Memory utilization (MB)	Throughput (ops/sec)	Accuracy (%)
100	22.60	45.02	89.54	44.24	98.50
500	31.41	64.47	121.39	31.83	98.38
1,000	34.63	72.83	138.77	28.88	98.27
5,000	42.62	80.73	172.07	23.46	98.15
10,000	45.83	92.18	189.01	21.82	98.03

limits, indicating that the system maintains a high level of efficiency even at larger scales. The stability of throughput further supports the scalability of the method, as it does not exhibit significant degradation that could hinder real-world usability.

Importantly, the accuracy of the method remains consistently high across all tested network sizes, confirming its robustness in large-scale scenarios. For 100 nodes, accuracy is 98.50%, and as the network size increases, accuracy remains relatively stable at 98.27% for 1000 nodes, 98.15% for 5000 nodes, and 98.03% for 10,000 nodes. The slight variations indicate that even as network complexity grows, the method maintains strong predictive performance without significant degradation. This implies that the proposed approach is well-suited for large-scale network applications. The logarithmic increase in execution time, coupled with controlled CPU and memory utilization, ensures that the system can handle increasing network sizes without excessive computational overhead. Additionally, the stability of throughput and accuracy further strengthens the feasibility of the method for deployment in real-world scenarios. The findings validate the scalability of the approach, reinforcing its applicability in large-scale distributed computing, IoT networks, and high-traffic communication systems.

5.8. Ablation study

The results of the ablation study clearly highlight the contributions of each component of the RAGN model to its overall robustness and effectiveness in detecting malicious traffic, particularly in the face of adversarial attacks. The removal of any one of the four major components – the dynamic attention mechanism, feature regularization, graph structure adjustment, and extra attention mechanism on edges – resulted in noticeable declines in performance, both in terms of accuracy and resilience to adversarial attacks. First, the ablation of the dynamic attention mechanism resulted in a significant accuracy drop (approximately 5%–7%) and a corresponding increase in the misclassification rate (by 10%). This confirms that the dynamic recalibration of attention scores based on real-time graph structure changes is vital for detecting subtle adversarial modifications to the graph structure. Without it, the model behaves similarly to static GAT architectures, which are less capable of adapting to new adversarial tactics.

The removal of feature regularization also led to a performance drop, with a decline in accuracy by around 4%–6%. The absence of regularization particularly affected robustness in noisy environments, where adversarial feature perturbations became more effective. The increase in the misclassification rate (8%–9%) further supports the idea that regularization is essential for smoothing out variations between neighboring nodes' features, making the model more resistant to feature noise. When the graph structure adjustment component was removed, the model's ability to withstand structural perturbations such as adversarial edge modifications was heavily compromised. The accuracy dropped by 6%–8%, and the misclassification rate increased by an average of 9%. This indicates that dynamically updating the adjacency matrix during training plays a key role in helping the model recognize and neutralize adversarial edges that could otherwise mislead the detection system.

The final component, extra attention on the edges, was also shown to be critical. Its removal caused a drop in accuracy by 5%–7%, particularly when dealing with adversarial attacks targeting the edges of the graph. The misclassification rate increased by around 11%,

Table 10
Results of ablation study results.

Ablation type	Dataset	Accuracy (%)	Misclassification Rate (%)	Accuracy (+)
No Dynamic Attention	CICIDS-2018	80.5	18.2	+7.3%
	CTU-13	79.8	19.1	+7.8%
	USTC-TFC2016	82.4	16.8	+6.2%
No Feature Regularization	CICIDS-2018	81.1	17.5	+6.7%
	CTU-13	80.3	18.6	+7.3%
	USTC-TFC2016	83.1	15.9	+5.9%
No Graph Structure Adjustment	CICIDS-2018	79.2	19.4	+8.1%
	CTU-13	78.7	20.2	+8.9%
	USTC-TFC2016	81.9	17.4	+6.7%
No Extra Edge Attention	CICIDS-2018	80.7	18.0	+7.1%
	CTU-13	80.1	18.8	+7.5%
	USTC-TFC2016	82.6	16.3	+6.3%

highlighting the importance of distinguishing between benign and adversarial edges. This extra attention mechanism effectively reduces the impact of adversarial manipulations that attempt to exploit the model's reliance on graph edges for node classification. Overall, the full RAGN model, incorporating all four components, consistently outperformed the ablated versions in terms of both detection accuracy and robustness to adversarial attacks. The results clearly demonstrate that the combination of dynamic graph adjustment, feature regularization, and adaptive attention mechanisms are essential for improving the model's performance. Specifically, the RAGN model achieves up to 8% higher accuracy compared to its ablated counterparts, reinforcing the effectiveness of the integrated design choices. (See Table 10.)

5.9. Application of RAGN and trade-offs

The Robust Adaptive Graph Network (RAGN) is designed for deployment in environments where the nature of attacks can evolve quickly, such as enterprise networks, critical infrastructure, and cloud-based systems. RAGN's dynamic capabilities make it particularly suitable for these high-risk settings, where attackers constantly refine their techniques to evade detection. The model is designed to analyze network traffic by constructing it as a dynamic graph, where entities like IP addresses, devices, and servers are represented as nodes, and their interactions (such as data transfers or connection attempts) are modeled as edges. This approach allows RAGN to capture complex relationships and dependencies within the network traffic, giving it a significant advantage over traditional static models. Unlike existing solutions, RAGN continuously refines both the graph structure and node features through an iterative process. This enables it to respond effectively to evolving threats, including zero-day attacks and adversarial manipulations. The iterative attention mechanism dynamically adjusts the importance of each edge in the graph, allowing RAGN to prioritize genuine traffic while reducing the influence of adversarial edges. As a result, the model maintains robustness even when attackers introduce subtle perturbations intended to deceive the detection system.

RAGN is particularly effective in detecting a wide range of attack types. For targeted attacks, where adversaries focus on specific nodes, such as critical servers, RAGN's dynamic attention mechanism recalibrates the graph to detect these threats, even when attackers attempt to obscure their activities by modifying traffic patterns. In contrast,

untargeted attacks, such as Distributed Denial of Service (DDoS) attacks, involve overwhelming the network with traffic. RAGN effectively identifies these large-scale disruptions by analyzing anomalies in the traffic graph, even when adversaries introduce random perturbations to evade detection. The model's robustness extends to random feature attacks, where an attacker manipulates the characteristics of individual nodes, such as packet size, connection duration, or protocol type. RAGN's use of Laplacian regularization ensures that these feature-level perturbations are smoothed across the graph, mitigating their impact on detection accuracy. This ability to filter out random feature noise allows RAGN to remain highly accurate, even in the presence of adversarial feature manipulations.

One of RAGN's key strengths lies in its ability to detect zero-day exploits. These are new, previously unknown vulnerabilities that attackers exploit before they are patched. RAGN's iterative refinement process allows it to detect deviations in network behavior, flagging novel attack patterns that other models might miss. This adaptability is crucial for environments where security threats are constantly evolving. Additionally, RAGN excels in identifying coordinated attacks, such as brute force and botnet activities, by analyzing repetitive patterns in the network graph. The model recognizes the repeated connection attempts typical of brute force attacks, as well as the synchronized behaviors of botnets, which involve multiple compromised devices acting in unison. RAGN's design makes it particularly effective in dynamic environments, such as cloud-based architectures where nodes are frequently added or removed. Its ability to adapt in real-time to changes in the network landscape ensures that it can effectively detect both emerging and evolving threats. While RAGN's iterative refinement process and dynamic attention mechanism introduce a trade-off in terms of computational complexity, the increased resource consumption is justified in high-risk environments. The iterative refinement of graph structures and features, coupled with the dynamic attention mechanism, requires additional computational resources compared to more traditional models. Another important trade-off is while RAGN excels in resisting targeted, zero-day attacks, and random attacks, this adaptability requires a more intricate training process. The iterative adjustments and attention re-weighting, though highly effective in enhancing robustness, could pose a challenge in time-sensitive applications or environments with limited computational resources. Therefore, while RAGN offers superior defense capabilities, organizations must assess whether their operational environment justifies the increased complexity and computational demands.

Deploying deep learning-based intrusion detection systems in large-scale network environments presents significant privacy challenges, as data from multiple sources must be processed while ensuring user confidentiality and system security. In our approach, RAGN constructs graph-based representations of network traffic, inherently minimizing exposure to raw packet payloads to mitigate privacy risks. However, large-scale networks often involve distributed data from various entities, making them vulnerable to data leakage, inference attacks, and unauthorized access. Future research could explore the integration of privacy-preserving techniques with RAGN to further enhance security and compliance with privacy regulations. Specifically, incorporating federated learning, data anonymization, differential privacy, homomorphic encryption, and blockchain-based access control mechanisms could strengthen the privacy protection of RAGN in large-scale networks.

6. Conclusion

In this study, we introduced the Robust Adaptive Graph Network (RAGN), a novel framework designed to address critical vulnerabilities in Graph Attention Networks (GATs) for detecting malicious network traffic, particularly in dynamic and adversarial environments. RAGN incorporates key innovations, including a dynamic attention mechanism and iterative refinement of graph structures and node features,

guided by feature smoothness regularization. These advancements enable RAGN to adapt in real-time to adversarial changes, prioritize reliable connections, and mitigate the impact of adversarial manipulations, significantly enhancing its robustness and accuracy.

We also proposed the Semantic-Preserving Adversarial Node Injection (SPAN), a benchmark tailored for evaluating the resilience of dynamic graph neural networks (DGNNs) under adversarial conditions. By exploiting structural vulnerabilities and leveraging dynamic graph characteristics, SPAN provides a rigorous evaluation framework for testing RAGN and other models in realistic attack scenarios.

Our extensive experiments on four real-world datasets – CICIDS-2018, CTU-13, USTC-TFC2016, and UJS-IDS2022 – demonstrate that RAGN consistently outperforms state-of-the-art models such as GCN-MHSA, FedGAT, and GATrans across multiple metrics. On UJS-IDS2022, RAGN achieved a detection accuracy of 98.12% and an F1-score exceeding 97%, while maintaining a false positive rate as low as 0.22%. These results highlight RAGN's ability to detect evolving attacks and adapt to adversarial manipulations. Moreover, RAGN's performance improvement is directly tied to its innovations, such as the dynamic attention mechanism, which enables more effective weighting of edges, and iterative refinement, which reduces the influence of adversarial nodes and features. Compared to baseline models that struggle with feature perturbations or graph structure manipulation, RAGN demonstrates higher resilience and adaptability, achieving significantly lower misclassification rates even in high-perturbation scenarios. Beyond malicious traffic detection, RAGN sets a new benchmark for graph neural networks in high-stakes environments. While our method introduces mechanisms to enhance robustness, future work could explore hybrid approaches that combine privacy-preserving techniques, real-time streaming architectures, and scalable distributed frameworks to ensure that intrusion detection remains both effective and practical in large-scale, high-throughput networks.

CRedit authorship contribution statement

Ernest Akpaku: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Jinfu Chen:** Writing – review & editing, Supervision, Methodology, Funding acquisition. **Mukhtar Ahmed:** Writing – review & editing, Conceptualization. **Francis Kwadzo Agbenyegah:** Writing – review & editing, Conceptualization. **William Leslie Brown-Acquaye:** Writing – review & editing, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Jinfu Chen reports financial support was provided by National Natural Science Foundation of China. Jinfu Chen reports financial support was provided by Natural Science Foundation of Jiangsu Province, China. Jinfu Chen reports financial support was provided by China Postdoctoral Science Foundation. Jinfu Chen reports financial support was provided by Qinglan Project of Jiangsu Province, China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partly supported by the National Natural Science Foundation of China (NSFC) (Grant nos. 62172194, 62202206 and U1836116), the Natural Science Foundation of Jiangsu Province, China (Grant no. BK20220515), the China Postdoctoral Science Foundation (Grant no. 2021M691310), and Qinglan Project of Jiangsu Province, China.

Data availability

Data will be made available on request.

References

- [1] M. Ahmed, J. Chen, E. Akpaku, R.N.A. Sosu, A. Latif, DELM: Deep ensemble learning model for anomaly detection in malicious network traffic-based adaptive feature aggregation and network optimization, *ACM Trans. Priv. Secur.* 27 (4) (2024) <http://dx.doi.org/10.1145/3690637>, URL <https://doi.org/10.1145/3690637>.
- [2] W. Wang, Y. Shang, Y. He, Y. Li, J. Liu, BotMark: Automated botnet detection with hybrid analysis of flow-based and graph-based traffic behaviors, *Inform. Sci.* 511 (2020) 284–296, <http://dx.doi.org/10.1016/j.ins.2019.09.024>, URL <https://www.sciencedirect.com/science/article/pii/S0020025519308758>.
- [3] L. Zhang, L. Tan, H. Shi, H. Sun, W. Zhang, Malicious traffic classification for IoT based on graph attention network and long short-term memory network, in: 2023 24th Asia-Pacific Network Operations and Management Symposium, APNOMS, 2023, pp. 54–59.
- [4] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, 2018, URL <https://arxiv.org/abs/1710.10903>, arXiv:1710.10903.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS '17, Curran Associates Inc., Red Hook, NY, USA, 2017, pp. 6000–6010.
- [6] T. Nandy, R. Md Noor, R. Kolandaisamy, M.Y.I. Idris, S. Bhattacharyya, A review of security attacks and intrusion detection in the vehicular networks, *J. King Saud Univ. - Comput. Inf. Sci.* 36 (2) (2024) 101945, <http://dx.doi.org/10.1016/j.jksuci.2024.101945>, URL <https://www.sciencedirect.com/science/article/pii/S131915782400034X>.
- [7] W. Jin, Y. Ma, X. Liu, X. Tang, S. Wang, J. Tang, Graph structure learning for robust graph neural networks, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 66–74, <http://dx.doi.org/10.1145/3394486.3403049>, URL <https://doi.org/10.1145/3394486.3403049>.
- [8] J. Yuan, H. Yu, M. Cao, J. Song, J. Xie, C. Wang, Self-supervised robust graph neural networks against noisy graphs and noisy labels, *Appl. Intell.* 53 (21) (2023) 25154–25170, <http://dx.doi.org/10.1007/s10489-023-04836-6>, URL <https://doi.org/10.1007/s10489-023-04836-6>.
- [9] Z. Li, F. Yuan, Y. Liu, C. Cao, F. Fang, J. Tan, Heterogeneous graph attention network for malicious domain detection, in: E. Pimenidis, P. Angelov, C. Jayne, A. Papaleonidas, M. Aydin (Eds.), *Artificial Neural Networks and Machine Learning – ICANN 2022*, Springer Nature Switzerland, Cham, 2022, pp. 506–518.
- [10] J. Chen, H. Xie, S. Cai, L. Song, B. Geng, W. Guo, GCN-MHSA: A novel malicious traffic detection method based on graph convolutional neural network and multi-head self-attention mechanism, *Comput. Secur.* 147 (2024) 104083, <http://dx.doi.org/10.1016/j.cose.2024.104083>, URL <https://www.sciencedirect.com/science/article/pii/S0167404824003882>.
- [11] J. Zheng, Z. Zeng, T. Feng, GCN-ETA: High-efficiency encrypted malicious traffic detection, *Secur. Commun. Netw.* 2022 (1) (2022) 4274139, <http://dx.doi.org/10.1155/2022/4274139>, URL <https://onlinelibrary.wiley.com/doi/abs/10.1155/2022/4274139>.
- [12] J. Yang, X. Jiang, Y. Lei, W. Liang, Z. Ma, S. Li, MTSecurity: Privacy-preserving malicious traffic classification using graph neural network and transformer, *IEEE Trans. Netw. Serv. Manag.* 21 (3) (2024) 3583–3597, <http://dx.doi.org/10.1109/TNSM.2024.3383851>.
- [13] F. Zola, L. Seguro-Gil, J. Bruse, M. Galar, R. Orduna-Urrutia, Network traffic analysis through node behaviour classification: A graph-based approach with temporal dissection and data-level preprocessing, *Comput. Secur.* 115 (2022) 102632, <http://dx.doi.org/10.1016/j.cose.2022.102632>, URL <https://www.sciencedirect.com/science/article/pii/S0167404822000311>.
- [14] S. Chen, B. Lang, H. Liu, Y. Chen, Y. Song, Android malware detection method based on graph attention networks and deep fusion of multi-modal features, *Expert Syst. Appl.* 237 (2024) 121617, <http://dx.doi.org/10.1016/j.eswa.2023.121617>, URL <https://www.sciencedirect.com/science/article/pii/S095741742302119X>.
- [15] C. Catal, H. Gunduz, A. Ozcan, Malware detection based on graph attention networks for intelligent transportation systems, *Electronics* 10 (20) (2021) <http://dx.doi.org/10.3390/electronics10202534>, URL <https://www.mdpi.com/2079-9292/10/20/2534>.
- [16] X. Han, S. Cui, J. Qin, S. Liu, B. Jiang, C. Dong, Z. Lu, B. Liu, ContraMTD: An unsupervised malicious network traffic detection method based on contrastive learning, in: Proceedings of the ACM Web Conference 2024, WWW '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 1680–1689, <http://dx.doi.org/10.1145/3589334.3645479>, URL <https://doi.org/10.1145/3589334.3645479>.
- [17] Z. Niu, J. Xue, D. Qu, Y. Wang, J. Zheng, H. Zhu, A novel approach based on adaptive online analysis of encrypted traffic for identifying malware in IIoT, *Inform. Sci.* 601 (2022) 162–174, <http://dx.doi.org/10.1016/j.ins.2022.04.018>, URL <https://www.sciencedirect.com/science/article/pii/S0020025522003565>.
- [18] W.W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, M. Portmann, E-GraphSAGE: A graph neural network based intrusion detection system for IoT, in: NOMS 2022–2022 IEEE/IFIP Network Operations and Management Symposium, IEEE Press, 2022, pp. 1–9, <http://dx.doi.org/10.1109/NOMS54207.2022.9789878>.
- [19] X. Luo, Y. Li, H. Cheng, L. Yin, AGCN-domain: Detecting malicious domains with graph convolutional network and attention mechanism, *Mathematics* 12 (5) (2024) <http://dx.doi.org/10.3390/math12050640>, URL <https://www.mdpi.com/2227-7390/12/5/640>.
- [20] C. Wang, G. Liu, From anomaly detection to classification with graph attention and transformer for multivariate time series, *Adv. Eng. Inform.* 60 (2024) 102357, <http://dx.doi.org/10.1016/j.aei.2024.102357>, URL <https://www.sciencedirect.com/science/article/pii/S1474034624000053>.
- [21] Z. Sun, A.M. Teixeira, S. Toor, GNN-IDS: Graph neural network based intrusion detection system, in: Proceedings of the 19th International Conference on Availability, Reliability and Security, ARES '24, Association for Computing Machinery, New York, NY, USA, 2024, <http://dx.doi.org/10.1145/3664476.3664515>, URL <https://doi.org/10.1145/3664476.3664515>.
- [22] R. Abu Bakar, L. De Marinis, F. Cugini, F. Paolucci, FTG-Net-E: A hierarchical ensemble graph neural network for DDoS attack detection, *Comput. Netw.* 250 (2024) 110508, <http://dx.doi.org/10.1016/j.comnet.2024.110508>, URL <https://www.sciencedirect.com/science/article/pii/S1389128624003402>.
- [23] I.J. King, X. Shu, J. Jang, K. Eykholt, T. Lee, H.H. Huang, EdgeTorrent: Real-time temporal graph representations for intrusion detection, in: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, RAID '23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 77–91, <http://dx.doi.org/10.1145/3607199.3607201>, URL <https://doi.org/10.1145/3607199.3607201>.
- [24] X. Han, S. Liu, J. Liu, B. Jiang, Z. Lu, B. Liu, ECNet: Robust malicious network traffic detection with multi-view feature and confidence mechanism, *IEEE Trans. Inf. Forensics Secur.* 19 (2024) 6871–6885, <http://dx.doi.org/10.1109/TIFS.2024.3426304>.
- [25] D. Zügner, A. Akbarnejad, S. Günnemann, Adversarial attacks on neural networks for graph data, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '18, Association for Computing Machinery, New York, NY, USA, 2018, pp. 2847–2856, <http://dx.doi.org/10.1145/3219819.3220078>, URL <https://doi.org/10.1145/3219819.3220078>.
- [26] H. Dai, H. Li, T. Tian, X. Huang, L. Wang, J. Zhu, L. Song, Adversarial attack on graph structured data, in: J. Dy, A. Krause (Eds.), Proceedings of the 35th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, vol. 80, PMLR, 2018, pp. 1115–1124, URL <https://proceedings.mlr.press/v80/dai18b.html>.
- [27] D. Zügner, O. Borchert, A. Akbarnejad, S. Günnemann, Adversarial attacks on graph neural networks: Perturbations and their patterns, *ACM Trans. Knowl. Discov. Data* 14 (5) (2020) <http://dx.doi.org/10.1145/3394520>, URL <https://doi.org/10.1145/3394520>.
- [28] X. Yang, S. Ruan, J. Li, Y. Yue, B. Sun, TrafCL: Robust encrypted malicious traffic detection via contrastive learning, in: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 2910–2919, <http://dx.doi.org/10.1145/3627673.3679839>, URL <https://doi.org/10.1145/3627673.3679839>.
- [29] L. Zhang, H. Shi, K. Zhang, H. Sun, W. Zhang, GraphMal: A network malicious traffic identification method based on graph neural network, in: 2023 International Conference on Networking and Network Applications, NaNA, 2023, pp. 262–267, <http://dx.doi.org/10.1109/NaNA60121.2023.00051>.
- [30] F. Capobianco, R. George, K. Huang, T. Jaeger, S. Krishnamurthy, Z. Qian, M. Payer, P. Yu, Employing attack graphs for intrusion detection, in: Proceedings of the New Security Paradigms Workshop, NSPW '19, Association for Computing Machinery, New York, NY, USA, 2020, pp. 16–30, <http://dx.doi.org/10.1145/3368860.3368862>, URL <https://doi.org/10.1145/3368860.3368862>.
- [31] G. Ren, G. Cheng, N. Fu, Accurate encrypted malicious traffic identification via traffic interaction pattern using graph convolutional network, *Appl. Sci.* 13 (3) (2023) <http://dx.doi.org/10.3390/app13031483>, URL <https://www.mdpi.com/2076-3417/13/3/1483>.
- [32] Z. Li, X. Cheng, L. Sun, J. Zhang, B. Chen, A hierarchical approach for advanced persistent threat detection with attention-based graph neural networks, *Secur. Commun. Netw.* 2021 (1) (2021) 9961342, <http://dx.doi.org/10.1155/2021/9961342>, arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1155/2021/9961342.
- [33] G. Hu, X. Xiao, M. Shen, B. Zhang, X. Yan, Y. Liu, TCGNN: Packet-grained network traffic classification via graph neural networks, *Eng. Appl. Artif. Intell.* 123 (2023) 106531, <http://dx.doi.org/10.1016/j.engappai.2023.106531>, URL <https://www.sciencedirect.com/science/article/pii/S0952197623007157>.

- [34] S. Zhang, Z. Zhou, D. Li, Y. Zhong, Q. Liu, W. Yang, S. Li, Attributed heterogeneous graph neural network for malicious domain detection, in: 2021 IEEE 24th International Conference on Computer Supported Cooperative Work in Design, CSCWD, 2021, pp. 397–403, <http://dx.doi.org/10.1109/CSCWD49262.2021.9437852>.
- [35] E. Abbe, Community detection and stochastic block models: Recent developments, *J. Mach. Learn. Res.* 18 (177) (2018) 1–86, URL <http://jmlr.org/papers/v18/16-480.html>.
- [36] J. Ma, S. Ding, Q. Mei, Towards more practical adversarial attacks on graph neural networks, in: H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, H. Lin (Eds.), *Advances in Neural Information Processing Systems*, Vol. 33, Curran Associates, Inc., 2020, pp. 4756–4766, URL https://proceedings.neurips.cc/paper_files/paper/2020/file/32bb90e8976aab5298d5da10fe66f21d-Paper.pdf.
- [37] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, 2017, URL <https://arxiv.org/abs/1609.02907>, arXiv:1609.02907.
- [38] M. Wang, L. Yu, D. Zheng, Q. Gan, Y. Gai, Z. Ye, M. Li, J. Zhou, Q. Huang, C. Ma, Z. Huang, Q. Guo, H. Zhang, H. Lin, J. Zhao, J. Li, A.J. Smola, Z. Zhang, Deep graph library: Towards efficient and scalable deep learning on graphs, 2019, CoRR, arXiv:1909.01315, arXiv:1909.01315.
- [39] I. Sharafaldin, A.H. Lashkari, A.A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: *International Conference on Information Systems Security and Privacy*, 2018, URL <https://api.semanticscholar.org/CorpusID:4707749>.
- [40] S. García, M. Grill, J. Stiborek, A. Zunino, An empirical comparison of botnet detection methods, *Comput. Secur.* 45 (2014) 100–123, <http://dx.doi.org/10.1016/j.cose.2014.05.011>, URL <https://www.sciencedirect.com/science/article/pii/S0167404814000923>.
- [41] W. Wang, M. Zhu, X. Zeng, X. Ye, Y. Sheng, Malware traffic classification using convolutional neural network for representation learning, in: 2017 International Conference on Information Networking, ICOIN, 2017, pp. 712–717, <http://dx.doi.org/10.1109/ICOIN.2017.7899588>.
- [42] W. Jianping, Q. Guangqiu, W. Chunming, J. Weiwei, J. Jiahe, Federated learning for network attack detection using attention-based graph neural networks, *Sci. Rep.* 14 (1) (2024) 19088.
- [43] Z. Diao, G. Xie, X. Wang, R. Ren, X. Meng, G. Zhang, K. Xie, M. Qiao, EC-GCN: A encrypted traffic classification framework based on multi-scale graph convolution networks, *Comput. Netw.* 224 (2023) 109614, <http://dx.doi.org/10.1016/j.comnet.2023.109614>, URL <https://www.sciencedirect.com/science/article/pii/S1389128623000592>.
- [44] M. Waniek, T.P. Michalak, T. Rahwan, M. Wooldridge, Hiding individuals and communities in a social network, *Nat. Hum. Behav.* 2 (2016) 139–147, URL <https://api.semanticscholar.org/CorpusID:11496208>.
- [45] Y. Jiang, H. Xia, Adversarial attacks against dynamic graph neural networks via node injection, *High-Confid. Comput.* 4 (1) (2024) 100185, <http://dx.doi.org/10.1016/j.hcc.2023.100185>, URL <https://www.sciencedirect.com/science/article/pii/S2667295223000831>.
- [46] H. Wang, J. Wang, J. Wang, M. Zhao, W. Zhang, F. Zhang, X. Xie, M. Guo, GraphGAN: Graph representation learning with generative adversarial nets, *Proc. the AAAI Conf. Artif. Intell.* 32 (1) (2018) <http://dx.doi.org/10.1609/aaai.v32i1.11872>, URL <https://ojs.aaai.org/index.php/AAAI/article/view/11872>.
- [47] F. Cao, Q. Chen, H. Ye, An effective targeted label adversarial attack on graph neural networks by strategically allocating the attack budget, *Knowl.-Based Syst.* 293 (2024) 111689, <http://dx.doi.org/10.1016/j.knosys.2024.111689>, URL <https://www.sciencedirect.com/science/article/pii/S0950705124003241>.



Ernest Akpaku holds an M.Phil. in Management Information Systems from the University of Ghana. He is currently pursuing a Ph.D. in Computer Science and Technology at the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. His research interests encompass network security, malicious network traffic detection, vulnerability detection, and deep learning. Ernest is also a professional member of the Association for Computing Machinery (ACM).



Jinfu Chen received his Ph.D. degree in Computer Science and Technology from Huazhong University of Science and Technology, Wuhan, China, in 2009. He is currently a full professor at the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. His major research interests include Software Testing, Software Security, and Trusted Software. He has published more than 80 papers in some famous journals or conferences.



Mukhtar Ahmed received his BSc. degree in Computer Science from BUITEMS, Pakistan, in 2011, and the MS degree in Computer Science from ILMA University, Pakistan, in 2021. He is currently pursuing a Ph.D. degree at the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. His research interests include malicious network detection, network security, deep learning, and cloud security. He is a professional member of the ACM.



Francis Kwadzo Agbenyegah holds a Ph.D. in Computer Science from Kwame Nkrumah University of Science and Technology, Kumasi, Ghana. He is currently pursuing a Ph.D. in Computer Science and Technology at the School of Computer Science and Communication Engineering, Jiangsu University, Zhenjiang, China. His research interests include software vulnerability prediction and detection, machine learning, and deep learning. Francis is also a member of the Internet Society.



William Leslie Brown-Acquaye is a Senior Lecturer and the Dean of the Faculty of Computing and Information Systems at Ghana Communication Technology University, Accra, Ghana. He holds a Ph.D. in Automation and Control from Tambov State Technical University, Russian Federation. His research interests include control systems, pervasive computing, and human-robot interaction.